Storage

→ The place where we will keep our data
→ the first storage device is invented by IBM

Storage Device:: the device which we store our data
EX:: Pendrives, Memory cards, Hard drive, Storage Array

Kilo Byte [KB] = 1024 Bytes
Mega Byte [MB] = 1024 KB's
GB = 1024 MB's
TB = 1024 GB's
PB = 1024 TB's
ZB = 1024 PB's

Types of Storage ::

① primary storage
② secondary storage

① primary Storage ::
the storage which is available with the laptops / server is called primary storage
→ It is also known as main memory (storage space)
→ we will install OS on primary storage devices
EX:: Internal storage space of Mobiles, Internal hard disk, Boot drive

② secondary storage ::
the storage which we connect external to the laptop/server
EX:: External hard drive, JBOD, EBOD, Storage Array

① Local Storage
② Remote Storage

① Local Storage :: The storage space which is available locally with the system is called as local storage
EX:: Hard drives which is connected to laptop, DAS (Direct Attached Storage), Pendrive, JBOD

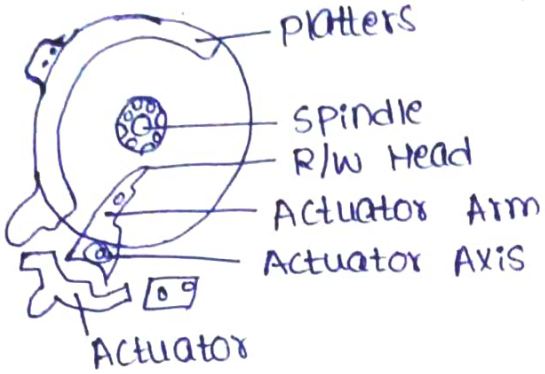
② Remote Storage :: The storage space which we can access remotely is called Remote storage
EX:: Google drive, one drive, SAN (Storage Area N/W) and NAS (Network Attached Storage)

Types of Drives::

① HDD [Hard drive disk]
② SSD [Solid State disk]

① HDD::

→ It stands for Hard disk drive
→ It works based on mechanical components



Components of HDD::

① platter
② spindle
③ Read/write Head
④ Actuator Arm
⑤ Actuator

① platter ::

→ It is a circular disk [rotating component] where user data will set
→ It is a rotating circular disk where actual data will be stored in HDD

② spindle ::

→ It is a motor inside the HDD which helps platter to rotate

③ Read / write Head ::

→ which is used to write the data in platter

④ Actuator ::

→ Actuator receives data from host and send it to platter

How data writes in HDD::

→ In HDD data stores in platters in a form of sectors, Group of sectors called as blocks



→ we can define our own block size
→ the standard block size is 4K
→ HDD is less expensive
→ HDD's are more reliable [As they use mechanical components]
→ life span of HDD is more
→ form factor of HDD is 3.5 inches

SSD (Solid State Drive)::

→ SSD stands for solid state drive
→ It works based on flash memory

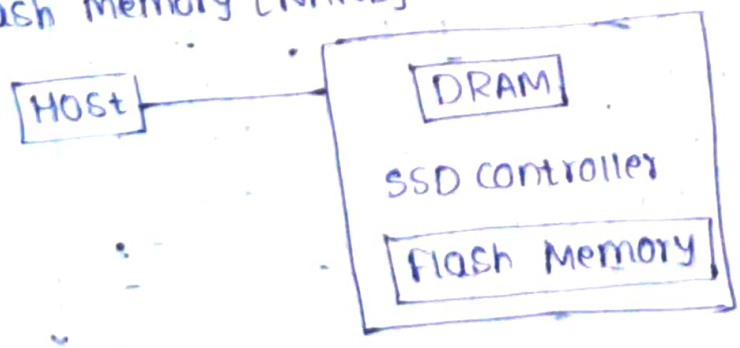
Types of flash Memory
① NAND
② NOR

→ SSD's uses NAND based flash memory

components of SSD::

① DRAM [cache]
② SSD controller
③ Flash memory [NAND]

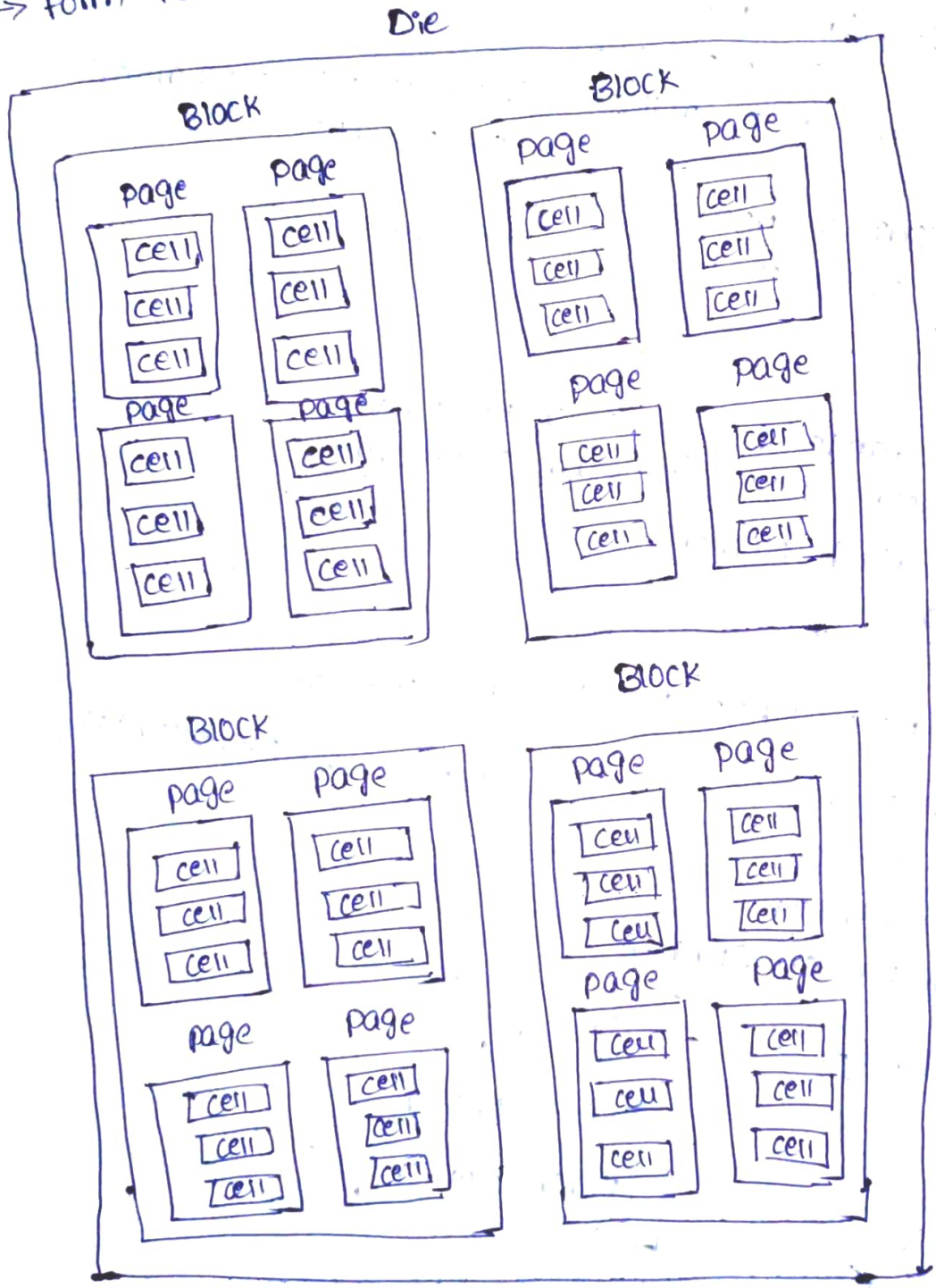
→ PLP(Power loss protection)::



How SSD works::

→ In SSD data stores in flash memory
→ from host data comes to DRAM [cache memory]

→ first the data will be written in a DRAM [cache memory]
→ SSD controller takes the data from DRAM and writes in flash memory [NAND]
→ In NAND data stores in form of cells
→ Group of cells is called as pages
→ Group of pages are called as Blocks
→ Group of Blocks are called as Die
→ Lifespan of SSD is less than HDD
→ form factor of SSD is 2.5 inches, M.2, U.2, E1.S, E2.S, E3.S



Difference b/w HDD and SSD

HDD	SSD
→ HDD stands for Hard disk drive	→ SSD stands for Solid state drive
→ HDD data stores in platters	→ SSD stores in flash Memory [NAND]
→ In HDD data stores in form of sectors / Blocks	→ In SSD data stores in form of cells / pages / Blocks
→ form factor of HDD is 3.5 inches	→ form factor of SSD is 2.5 inches
→ lifespan of HDD is more than SSD	→ Lifespan of SSD is less than HDD
→ HDD's are more reliable	→ SSD's are less reliable
→ HDD's are less expensive	→ SSD's are more costly
→ HDD's is slower than SSD	→ SSD is faster than HDD
→ In HDD we can overwrite the data	→ In SSD we need to erase the data before rewrite.
→ In HDD "write" operation happens in sector level	→ In SSD "write" operation happens in page level and "erase" happens in block level.

Types of NAND / SSD's

① SLC (Single Level Cell)
② MLC (Multi level cell)
③ TLC (Tri Level Cell)
④ QLC (Quadra Level Cell)
⑤ PLC (Penta Level Cell)

→ Performance, lifespan, costlier

① SLC (Single Level Cell) ::

→ SLC stands for Single level cell
→ It can store 1 bit per cell
→ SLC Drives are costlier than MLC, TLC and QLC drives
→ performance of SLC Drives is better than MLC, TLC and QLC drives
→ Lifespan is also more than MLC, TLC and QLC drives

cell
1 Bit

② MLC (Multi level cell) ::

→ MLC stands for Multi level cell
→ It can store 2 bits per cell

cell
2 Bits

→ MLC drives are less expensive than SLC Drives and more expensive than TLC and QLC drives
→ performance of MLC is less than SLC and more than TLC and QLC drives
→ Lifespan of MLC is less than SLC and more than TLC and QLC drives

③ TLC (Tri level cell):

→ TLC stands for Tri level cell
→ It can store 3 bits per cell

cell
3 bits

→ TLC drives are less expensive than SLC and MLC drives are more expensive than QLC drives
→ performance of TLC is less than SLC and MLC and more than QLC
→ Lifespan of TLC drives is less than SLC and MLC and more than QLC drives

④ QLC (Quadra level cell):

→ QLC stands for Quadra level cell
→ It can store 4 bits per cell

cell
4 Bits

→ QLC drives are cheaper than SLC, MLC and TLC drives
→ performance of QLC is less than SLC, MLC and TLC drives
→ Lifespan of QLC is less than SLC, MLC and TLC drives

RAID::

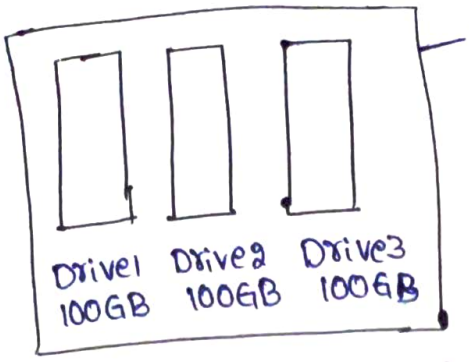
→ RAID Stands for Redundant Array of Independent disks

Array:: Group of Drives

Independent Drive :: Drive which is not Part of any RAID group

Redundant :: In case of one drive fails we can retrieve the data from another drives

→ RAID is a technology where we will apply RAID on top of the group of physical drives and create a logical drive called RAID Group



why we are going for RAID?

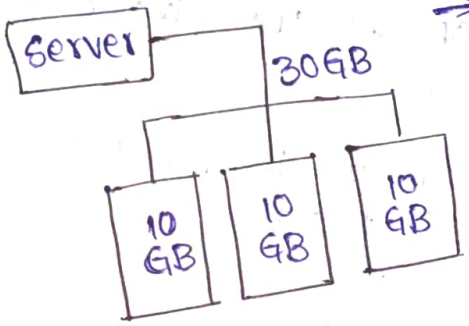
→ To achieve Redundancy (To retrieve the data in case of drive failures)

→ To avoid the data loss in case of drive failure

Ans: RAID Techniques::

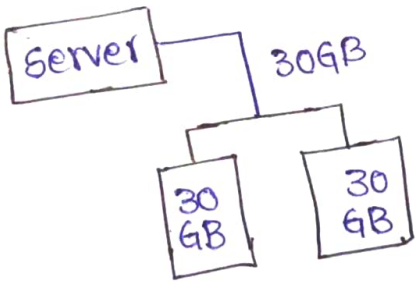
① Striping
② Mirroring
③ parity

① Striping :: Distributing the data equally all the drives in a RAID Group is called as striping



→ we cannot achieve redundancy

② Mirroring:: copying the same data to all the drives is called Mirroring

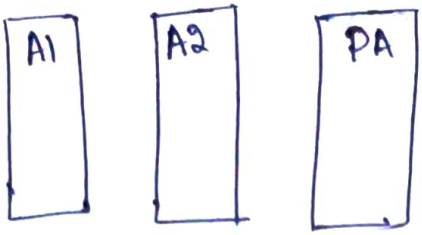


→ we can achieve redundancy

③ parity :: It calculates the difference b/w 2 drives and stores the output in another drive

XOR Method calculation ::

10 : 1
00 : 0
11 : 0
01 : 1



D1 D2 Parity Drive
1001 0101 D1 XOR D2

D1 :: 1001
D2 :: 0011
P1 :: 1010 D3 > D1 XOR D2

D1 fails :: D2 XOR D3 :: 0011
1010
1001 - D1

Types of parity ::

Dedicated parity ::

→ writing all the parity data in one drive is called Dedicated parity

Distributed parity ::

→ writing parity data on all the drives is called Distributed parity

Adv :: performance is better in distributed parity compared to dedicated parity

RAID Levels ::

RAID 0 > R0
RAID 1 > R1
RAID 3 > R3
RAID 4 > R4
RAID 5 > R5
RAID 6 > R6

Nested RAID Levels ::

RAID 0 + RAID 1 :: RAID 01 [R01]
RAID 1 + RAID 0 :: RAID 10 [R10]
RAID 5 + RAID 0 :: RAID 50 [R50]
RAID 6 + RAID 0 :: RAID 60 [R60]

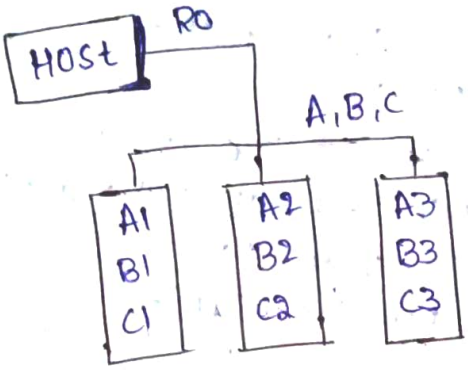
→ To overcome the limitations [Low performance / Redundancy] we are combining 2 RAID levels and creating nested RAID level

Fault Tolerance [FT] ::

→ No. of drives failures that RAID Level can with Stand

RAID 0 ::

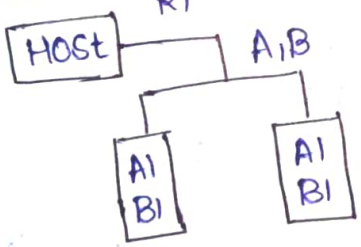
→ RAID 0 follows Striping technique
→ It requires minimum of 1 drive
→ It supports maximum drives upto 32 (vendor specific)
→ there is no redundancy in RAID 0, so RAID 0 is called as non-redundant RAID level and not an true RAID level
→ Fault tolerance of RAID 0 is zero.
→ Performance of R0 is better than R1
→ Redundancy of R0 is less than R1



→ Total available space = (no. of drives - F.T) * size of drive
[(N - FT) * size]
→ Total available space of R0 = N * size of the drive

RAID 1 ::

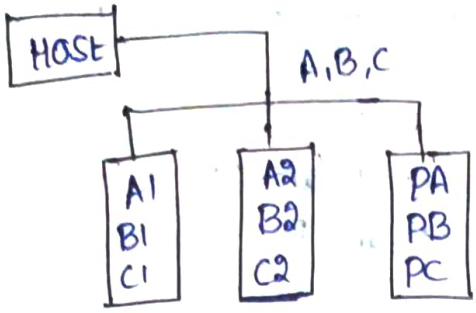
→ R1 follows mirroring techniques
→ whatever data comes from the host it will copy on all the drives



→ It requires minimum of 2 drives
→ It supports maximum drives also 2
→ It can with stand with 1 drive failure
→ Fault tolerance of R1 is 1.
→ 50% of drive space is getting wasted for mirroring
→ Total available space is N/2 * size of the drive
→ Performance of R1 is less than R0
→ Redundancy of R1 is better than R0.

RAID3::

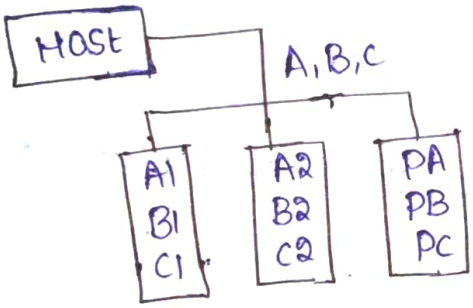
→ It follows Byte level stripping with dedicated parity
→ It requires minimum of 3 drives
→ It supports maximum drives upto 32



→ R3 can withstand with 1 drive failure
→ so fault tolerance of R3 is 1
→ Total available space is (N-1)* size of drive (N = NO. of drives and 1 = fault Tolerance)
→ performance of R3 is less than R4
→ Redundancy of R3 and R4 are same

RAID 4::

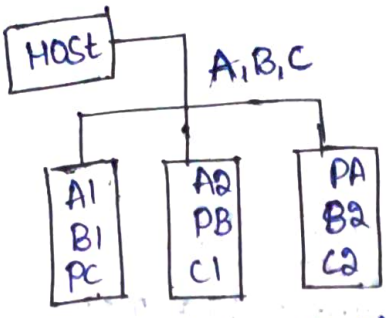
→ R4 follows block level striping with dedicated parity



→ It requires minimum of 3 drives
→ It supports maximum drives upto 32
→ It can with stand with 1 drive failure
→ fault tolerance of R4 is 1
→ Total available space is (N-1)* size of the drive.
→ performance of R4 is better than R3
→ Redundancy of R3 and R4 are same

RAID 5::

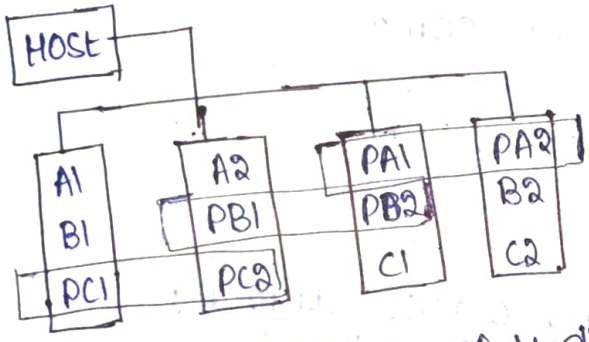
→ R5 follows Block level striping with ~~dedicated~~ parity distributed



→ It requires minimum of 3 drives
→ It supports maximum drives up to 32
→ It uses XOR algorithm to calculate parity
→ It can withstand with single drive failure
→ So, fault tolerance of RAID 5 is 1
→ Total available space = (N-1)* size of drive
→ R5 uses XOR algorithm for calculating the parity
→ performance of R5 is better than R6
→ Redundancy of R5 is less than R6

RAID 6::

→ R6 follows block level striping with dual distributed parity



XOR, Readsoloman

If we apply same algorithm there will be no diff in data means same data will be there. So that's why we use diff algorithms.

→ It requires minimum of 4 drives
→ It supports maximum drives up to 32 [vendor specific]
→ It can st with stand with 2 drives failure
→ So fault tolerance of RAID 6 is 2.
→ Total available space = (N-2)* size of drive
→ for first parity calculation R6 uses XOR Algorithm
→ for second parity calculation R6 uses Read soloman algorithm
→ Redundancy of R6 is better than R5
→ Performance of R6 is less than R5

Total Available Space

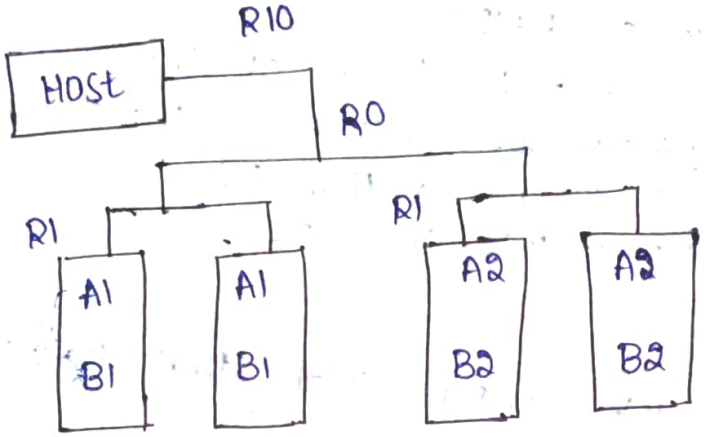
No. of drives - fault tolerance * size of the drive

100GB 100GB 100GB :: R0, R5

R0 :: (3-0) * 100 = 300 GB
R5 :: (3-1) * 100 = 200 GB

R10 :: [RAID 1 + RAID 0]

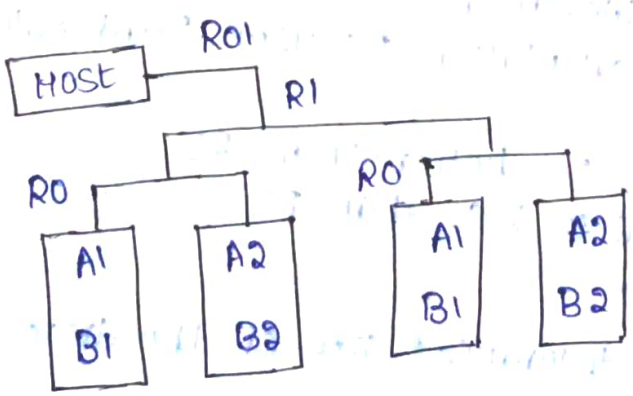
→ R10 follows stripping across mirroring
→ It requires minimum of 4 drives
→ It can with stand 2 drive failures but not from the same set
→ the fault tolerance of R10 is 2.
→ Total available space is (N-2) * size of the drive



→ performance of R10 is better than R01
→ Redundancy is will be same
→ To increase the performance of R1 we are combining R1 with R0

R01 :: [RAID 0 + RAID 1]

→ R01 follows mirroring across stripping
→ It requires minimum of 4 drives
→ It can with stand 2 drive failures but from the same set
→ the fault tolerance of R01 is 2
→ Total available space is (N-2) * size of the drive

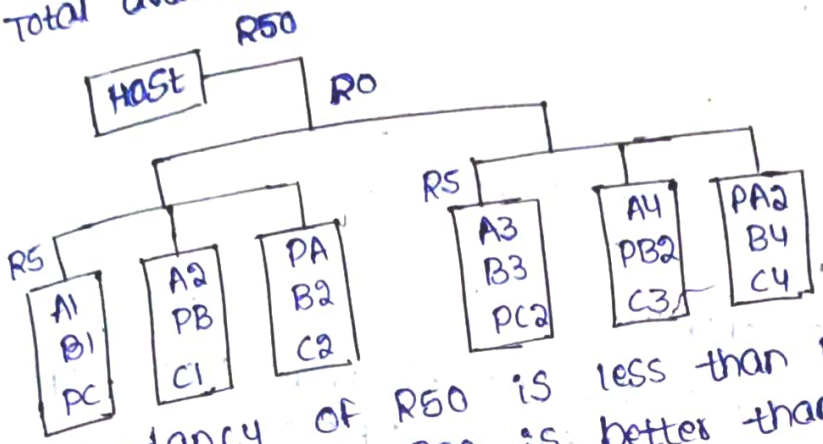


→ Performance of R01 is ~~better~~ less than R10.

→ Redundancy will be same

R50:: [RAID 5 + RAID 0]

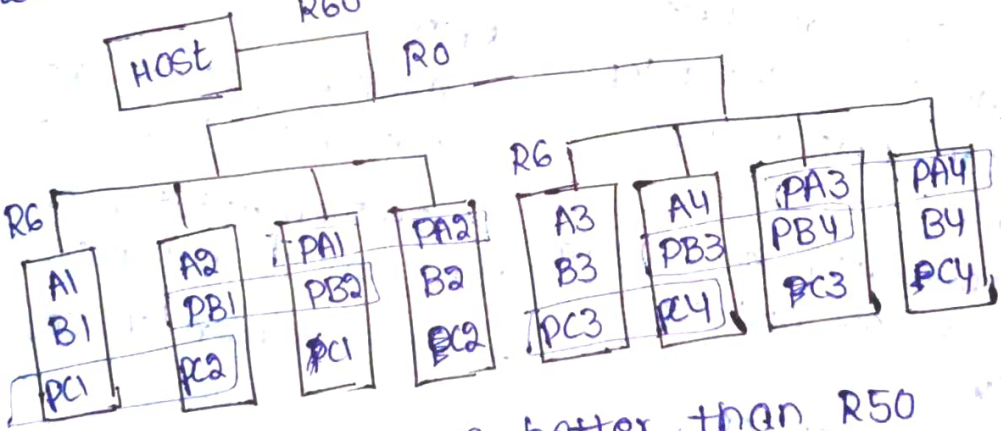
→ R50 follows stripping across R5 sets
→ R50 requires minimum of 6 drives and it will support maximum drives up to 32 (vendor specific)
→ R50 can with stand 2 drive failures not from the same set
→ the fault tolerance of R50 is 2
→ Total available space is (N-2)* size of the drive



→ Redundancy of R50 is less than R60
→ performance of R50 is better than R60

R60:: [RAID 6 + RAID 0]

→ R60 follows Stripping across R6 sets
→ R60 requires minimum of 8 drives and
→ R60 can with stand 4 drive failures not from the same set.
→ fault tolerance of R60 is 4.
→ Total available space is (N-4)* size of the drive.



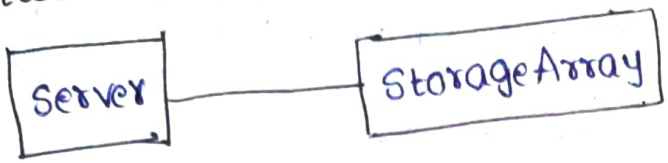
→ Redundancy of R60 is better than R50
→ Performance of R60 is less than R50

→ compared to all the RAID levels redundancy is better in R60 and performance is better in R0

Types of RAID::

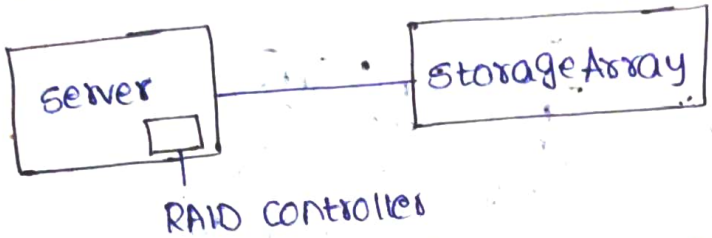
① Software RAID
② Hardware RAID

① Software RAID::



→ creation of RAID by using the operating system (OS) is called as software RAID.

② Hardware RAID::



→ creation of RAID by using the external device (RAID controller) is called as Hardware RAID.

How to create software RAID and Hardware RAID::

① Software RAID :: "mdadm" [TOOL]
② Hardware RAID :: "storcli" [TOOL]

Software RAID

→ creation of RAID by using the operating system (OS) is called as software RAID
→ software is OS dependent
→ software RAID is less reliable
→ Implementation of software RAID is simple and less expensive
→ More Burden on CPU
→ less performance
→ OS can discover drives directly

→ mdadm --create /dev/md0 --level=0 --raid-devices=2 /dev/sdc /dev/sdd

Hardware RAID

→ Creation of RAID by using the external device (RAID controller) is called as Hardware RAID.
→ Hardware is OS dependent
→ Hardware is more reliable
→ Implementation of Hardware is expensive
→ less Burden on CPU
→ more performance
→ Drives will be discovered by RAID controller

→ storcli /c0 add VD → virtual drive
→ controller0
type = raid1 drives = 252:0
raid level, enclosure ID, slot number

RAID States :: → Except R0 all drives are redundant drives

① Initialization ::
→ As soon as we create raid the state of the raid goes to initialization (converting physical drives into logical drives / RAID group)
→ we can't perform read / write operations in initialization state

② online ::
→ After initialization the state of the raid goes to online
→ we can perform read and write operations in online state

③ Degraded ::
→ If drive fails within the fault tolerance level in RAID level the state of RAID goes to Degraded
→ we can perform Read and write operations if the RAID is in degraded state
Example :: ① one drive failure in R5
② Two drive failure in R6

④ offline ::
→ If drive fail more than its fault tolerance level the state of the RAID goes to offline
→ we can not perform read and write operations if the RAID is in offline state,
Example :: ① one drive failure in R0
② Two drives failure in R5
③ Three drive failure in R6

⑤ Reconstruction / Rebuilding ::
→ After replacing failed drive with working drive the data in failed drive will start rebuilding [copying] in good drive then the state of the RAID goes to rebuilding / reconstruction
→ we can perform read and write operations if the RAID is in rebuilding state

⑥ copyback ::
→ once we replace failed drive to working drive the data in working drive copy back to newly working

drive the state of the drive goes to copy back

RAID features:

Hot spare :: ①

→ If we assign hot spare to the RAID group when ever drive fails in RAID group the hot spare drive will automatically becomes the part of RAID group

→ Assigning the spare drive to RAID group is called as hot spare

→ spare drive :: which is not part of any RAID Group

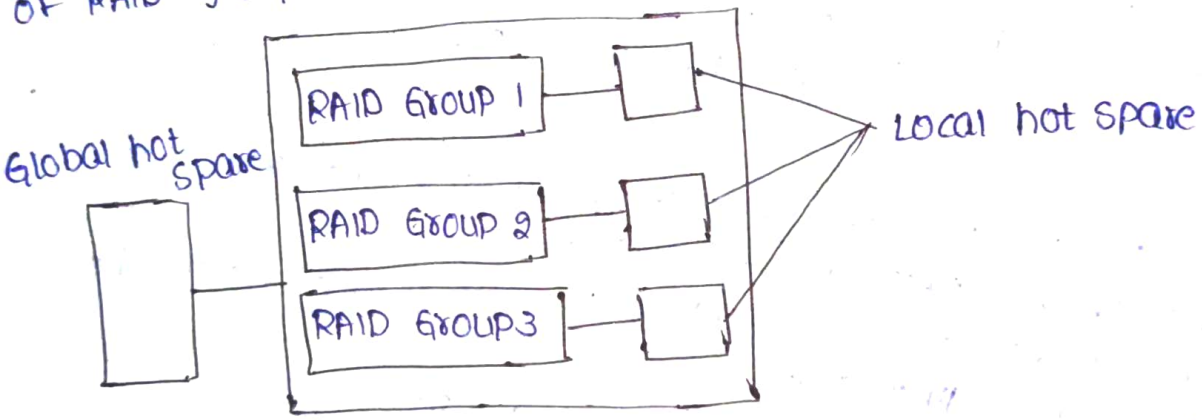
They are two types of hot spares

① local hot spare

② Global hot spare

① Local hot spare :: Assigning spare drive to the individual RAID group is called as local hot spare

② Global hot spare :: Assigning spare drive to the group of RAID group is called as Global hot spare



→ If drive fails and both local and global hot spares are assigned, local hot spare first becomes the part of RAID group

These will be performance drop in rebuild compare to online

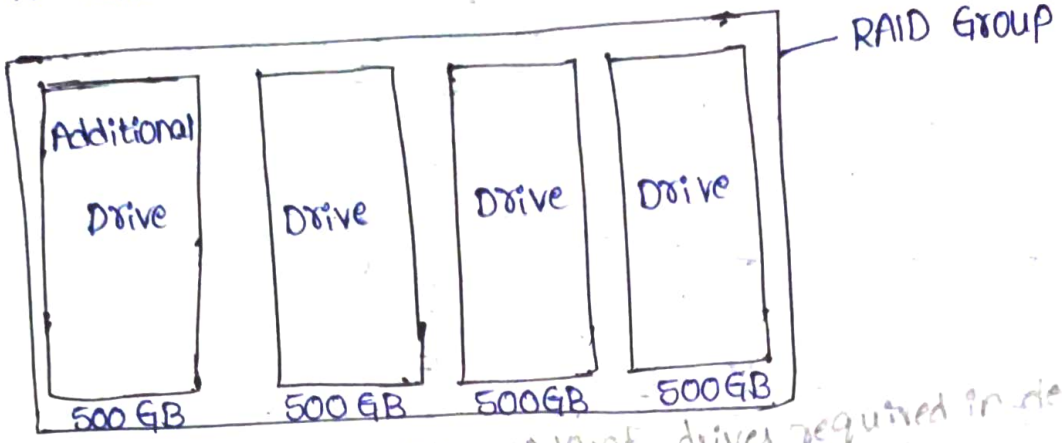
→ Performance - Read/write - online → more performance
- Rebuilding → less performance

S.NO	RAID'S	Minimum Drives	Maximum Drives	Performance	Redundancy	fault Tolerance	Total Available space
1.	R0	1	32	R0 is better than R1	R0 is less than R1	0	N* Size of the drive
2.	R1	2	2	R1 is less than R0	R1 is better than R0	1	N/2* Size of the drive
3.	R3	3	32	R3 is less than R4	R3 and R4 same	1	(N-1)* Size of the drive
4.	R4	3	32	R4 is better than R3	R3 and R4 same	1	(N-1)* Size of the drive
5.	R5	3	32	R5 is better than R6	R5 is less than R6	1	(N-1)* Size of the drive
6.	R6	4	32	R6 is less than R5	R6 is better than R5	2	(N-2)* Size of the drive
7.	R10	4	32	R10 is better than R01	R10 and R01 same	2	(N-2)* Size of the drive
8.	R01	4	32	R01 is less than R10	R01 and R10 same	2	(N-2)* Size of the drive
9.	R50	6	32	R50 is better than R60	R50 is less than R60	2	(N-2)* Size of the drive
10.	R60	8	32	R60 is less than R50	R60 is better than R50	4	(N-4)* Size of the drive

ORCE:: ②

→ It stands for online RAID capacity expansion

→ when RAID level is online state, we can increase the RAID capacity by adding additional drives to the RAID group this is called ORCE

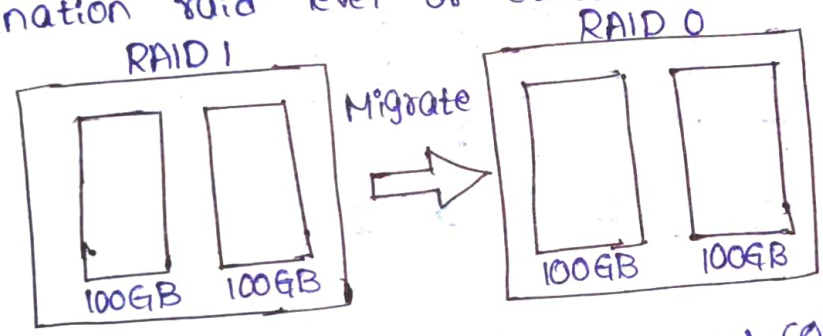


ORLM:: ③ total available space in source RAID level, No. of drives required in destination RAID level

→ ORLM stands for online RAID level Migration

→ when RAID level is in online state we can migrate RAID level from one RAID level to another supported RAID level based on the drive availability is called ORLM.

→ ORLM work based on No. of raids - total available space in source RAID level and the No. of drives required in destination raid level or other drive



④ consistency check :: [BCC] → Background ~~consistency~~ check consistency

→ consistency check is one of the feature of hardware ~~driv~~ raid

→ once we start consistency check it will verify the consistency data in RAID group

① To verify mirrored data (R1, R10, R01)

② To verify the parity inconsistency

③ If it finds any inconsistency in data it will fix the issues.

④ They are two modes in consistency check
   ⓐ verify only
   ⓑ verify and fix

Ex:: DELL :: perccli
Broadcom / Generic :: storcli / msm
Adaptec / microchip :: megacli
HP :: hpsascli

these are the tools used for checking the consistency check

⑤ Patrol Read ::

→ patrol Read is one of the feature of RAID controller
→ It will be applied on physical drives to detect and correct media errors

⑥ ECC ::

→ ECC stands for Error correction code
→ It is used to detect and correct parity Errors
   ① XOR
   ② Readsolomen

Types of IO [write and Read]

① Sequential IO :: filling the data block by block sequentially
   ⓐ sequential write
   ⓑ sequential Read

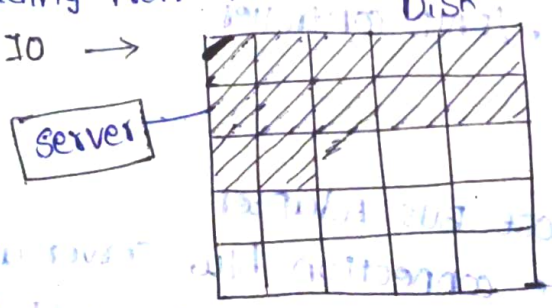
② Random IO :: Randomly filling the data block by block
   ⓐ Random write
   ⓑ Random Read

write :: writing to the drive
Read :: Reading to the drive

Sequential IO :: → sequential Io gives better performance compared to Random Io

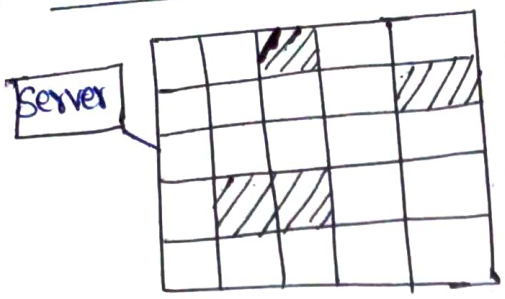
Read :: Reading from the drive

Sequential IO →



→ for sequential Io we use the variable block sizes
Ex:: 512 Bytes, 4KB, 4MB, 64MB
→ for Random Io we use the fixed block size of 4KB

Random IO::



Latency

→ Data comes from the users it convert to a cmd at host side and transfer to the target

Running IO from IO meter
: Launch IO meter application

Factors effects the performance of storage device::

① Block size
② Type of IO
③ Queuedepth

other side::

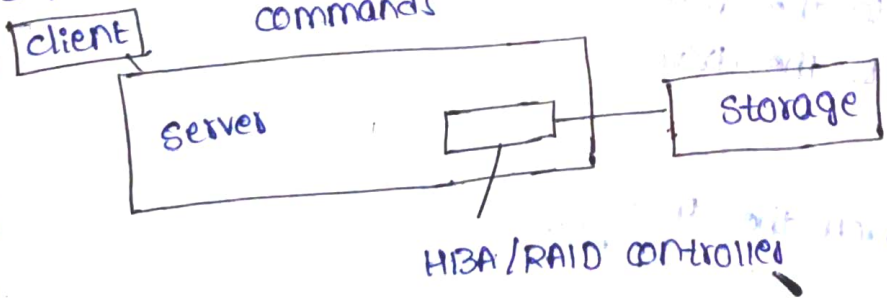
① IOPS - (Input output per second) - How many read/write operations a disk can do per second
It is measured in numbers
② Latency
③ throughput

Throughput::

→ At what speed the command is travelling to the target is called as throughput
→ It is measured in Numbers

Latency::

→ the time taken by a command to reach the target is called as latency
→ It is measured in mille seconds or micro seconds



HBA::

→ HBA stands for Host Bus Adapter
→ It establishes the connection b/w server and storage
→ we need to insert HBA in PCI slot of server motherboard

→ It takes the commands from users and send to the storage (target)

RAID controller ::

→ It works same as HBA; only difference is RAID controller comes with RAID features as default

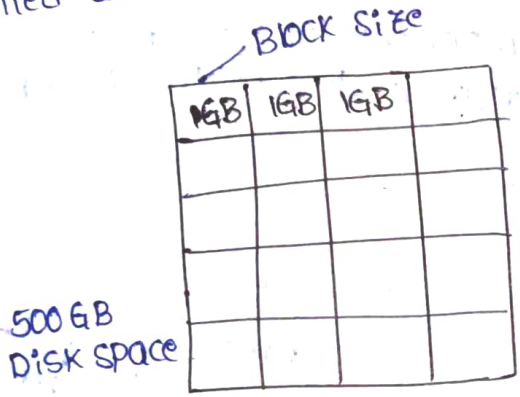
Queue depth ::

→ No. of outstanding commands that controller / HBA can hold at a time is called as queue depth

→ It is measured in Numbers.

Block size ::

→ It is amount of storage space. In drives the storage space is divided into No. of small chunks of data called as Block



HDD :: Sectors / Blocks
SSD :: cells / pages / Blocks

→ performance of storage system will depend on the block size queue depth

→ we need to give Block size and queue depth as a input to check the performance and we will measure IOPS, Latency and Throughput

High IOPS > more performance
High Latency > less performance
High Throughput > High performance

Different form factors ::

U.2, U.3 :: used in data center environments, hot swapable
M.2 :: used in desktop environments, not hot swapable

E1.S, E3.S → are for desktop environments [client devices]

E3.S
E1.L
E2.L

U.2, E3.S :: devices for datacenter environment
[enterprise storage devices]

Difference b/w HBA and RAID controller::
→ HBA and RAID controller are same
→ HBA :: used to establish communication b/w storage and server
RAID controller :: It is also for same purpose like used to establish communication b/w storage and server but it comes with default RAID feature

PCI ::
→ PCI stands for peripheral component interconnect
→ which is used to connect External devices to the CPU
→ Different types of PCI devices
① PCIX ② PCIE-E

Feature	PCI X	PCIE
full form	Extended	Express
Type	parallel	Serial → It is better because there is no data loss
Max Bandwidth	~1 GB/s	upto 128 GB/s
Scalability	limited	highly scalable
usage	legacy servers	Modern PCs, server storage, GPUs

→ PCI contains PCI lanes these are used to establish connection / transfer data b/w CPU and PCI

PCI lane ::
→ It is a communication channel b/w PCI and CPU
→ Each PCI lane contains multiple wires
so 1 wire is to transfer the data another wire is to receive the data

→ PCI is a full duplex protocol
→ Different PCI lanes
x1 → wifi / Network cards (single lane)
x4 → (four lanes) → SSD's
x8 → (eight lanes) → HBA / RAID controller / NIC
x16 → (sixteen lanes) → GPU
→ More number of lanes = More performance
→ there are different Generations of PCI

PCI Gen1	(PCIe 1.0)
PCI Gen2	(PCIe 2.0)
PCI Gen3	(PCIe 3.0)
PCI Gen4	(PCIe 4.0)
PCI Gen5	(PCIe 5.0)
PCI Gen6	(PCIe 6.0)

→ PCI Gen1 supports 2.5 GT/s (Gega transfer / seconds)
Each lane speed 250 MB/s
→ PCI Gen2 → 5 GT/s, Each lane speed 500 MB/s
→ PCI Gen3 → 8 GT/s, Each lane speed 985 MB/s
→ PCI Gen4 → 16 GT/s, " " " 1.97 GB/s
→ PCI Gen5 → 32 GT/s, " " " 3.94 GB/s
→ PCI Gen6 → 64 GT/s, " " " 7.88 GB/s
→ PCI Gen7 → 128 GT/s, " " "

PCI-E:
→ PCI-E stands for peripheral component Interconnect-Express
→ It is a channel b/w CPU and external peripheral devices like GPU, NVMe drives, RAID controllers, HBAs, Network cards
→ It supports point-to-point connection
→ PCIe supports serial communication and it is a full duplex protocol
→ Different PCIe slots :
x1, x4, x8, x16
→ we can insert x4 slot into x16 slot but we cannot insert x16 slot into x4 slot

Software RAID ::

"mdadm" :: to create md devices [SW RAID Device]

"LVM" > Logical Volume Manager

RAID ::

Drive1
Drive2 --> Logical Volume
Drive3

Name of the ~~driv~~ RAID
RAID level
RAID devices
BS = Block Size

"mdadm"

Syntax ::

mdadm --create <raid_name> --level=<RAID_Level>
--raid-devices=no.of devices <space> drive

mdadm --create /dev/md0 --level=0 --raid-devices=2
/dev/sdc /dev/sdd

Default Block Size = 512 KB
To customize Block Size = --chunk=<Block_Size>
To get details of RAID ::
Syntax :: mdadm --detail <raid_device_name>
Ex :: mdadm --detail /dev/md0
To Stop/delete SW RAID ::
Ex :: mdadm --Stop /dev/md0
To remove SW RAID ::
Ex :: mdadm --remove /dev/md0

To create R5 ::

mdadm --create /dev/md5 --level=5 --raid-devices=3
/dev/sdb /dev/sdc /dev/sdd

sdb :: 465 GB
sdc :: 279 GB
sdd :: 238 GB

R5 = N-1 * size of drive
= 3-1 * 238
= 2 * 238
= 476

RAID features ::
→ capacity Expansion [ORCE]
→ Hotspares
→ Migration

To add Disk to RAID Group ::
syntax :: mdadm --add <raid_group_name> <device_name>
EX :: mdadm --add /dev/md5 /dev/sde
→ we need to add same capacity drive in ORCE

To add hotspare drive ::
→ mdadm --add <raid_group> <device_name> --spare

Migration ::
syntax :: mdadm --grow <raid_device_name> --level=
<Target_raid_level>

Storage protocols ::

→ storage protocols will define the rules and standards that are required to establish storage connections
IEEE :: Institute of Electrical and Electronics Engineering
NAA :: National address Authority
SNIA :: Storage networking industry Association
Some of the storage protocols are ::

ATA
SATA
SCSI
SAS
ISCSi
FC
FCOE
NVme
NVme-OF

ATA ::

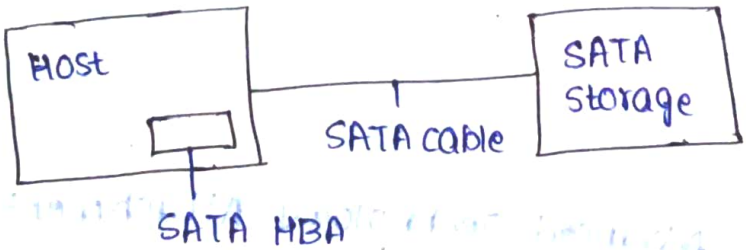
→ ATA stands for Advanced Technology Attachment
→ ATA is also called as IDE (Integrated Device Electronics)
→ ATA supports speed upto 300mbps (throughput)

→ ATA supports cable length up to 42cm
→ ATA supports up to 2 devices
→ ATA drives are identified by serial id
→ ATA is having its own command sets called as ATA commands

SATA::

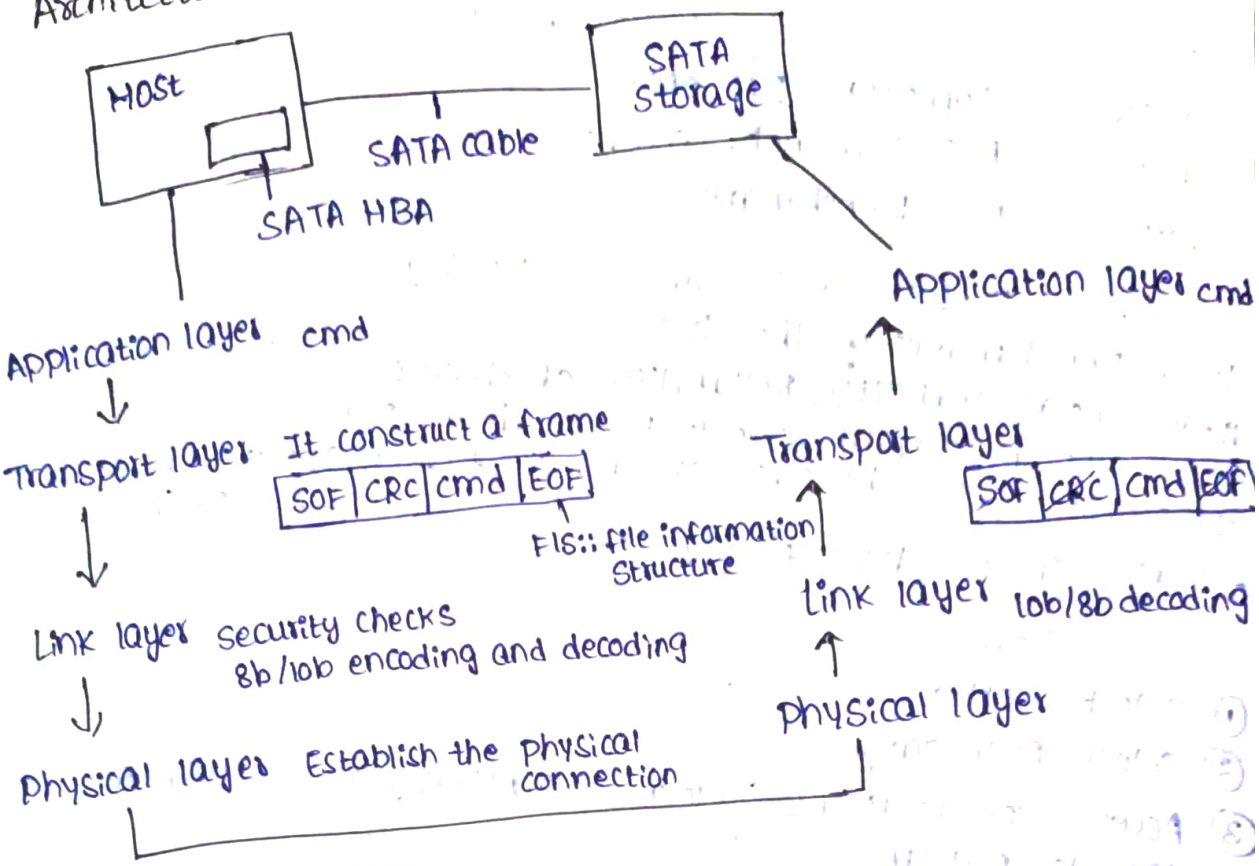
→ SATA stands for serial Advanced technology Attachment
→ SATA supports up to 8 devices
→ SATA supports speeds up to 6GBPS
→ SATA is a serial protocol
→ cable length of SATA is 1 meter
→ If we want to implement SATA storage we need to have SATA HBA, SATA cable
→ SATA is available as both HDD and SSD's
→ SATA devices are identified by serial ID

SATA storage



→ rpm of SATA is 7200
→ SATA supports single queue and it can hold upto 32 commands

Architecture of SATA ::



SOF :: Start of frame
EOF :: End of frame
CRC :: cyclic Redundancy check

Primitive ::

→ Application layer receives the data from host and sends to the transport layer

→ Transport layer receives the data from application layer and constructs the frame and sends to the link layer

→ Link layer receives the data from transport layer and performs 8b/10 encoding and decoding and sends the frame to physical layer

→ physical layer establishes the physical connections b/w host to target

Io :: Input/Output

Io Tools :: → crystal disk mark

- Windows :: Iometer, CDM etc..
- Linux :: FIO, Vdbench, IOzone
- ESXI :: HCI Benchmark

Use cases :: why we do performance testing

1. To Benchmarking the drive
2. To check stability / Reliability of drives

Iometer :: → we get benchmark sheet from developers
→ The pass criteria is +/-5%. results are accepted

→ used in windows to run Io

Steps in Iometer ::

① select worker
② select Disk Target
③ Access specifications [select Io pattern] or customize the Io pattern
④ Add Io pattern
⑤ Go to Test setup
   select
   ① Runtime
   ② Testname or Test description
⑥ start Io by clicking on start Flag

After starting Io, Results will be saved in .CSV file
Iometer will simulate the Io's and a file called iobw.tst will be created in the target drive

Total Io's per second :: Iops
Total MB/s per second :: throughput
Average Response time :: Latency
% CPU Utilization :: During Io, % of CPU utilized
Total error count :: If any Io errors, Io Timeouts, Io Delay

→ Performance testing on Storage device
① Raw drives ② file system formatted storage drives
③ on RAID volumes / RAID levels

FIO :: flexible IO (Windows & Linux)

→ used to simulate IO's in Linux Environment
→ short & drop must be '0' (zero) only

Inputs ::
- Iodepth
- BS
- Type of IO
- Drive Name
- Test Name
- Runtime
- NO. of Jobs
- IO Engine

fio --name=test_name --filename=<Drive_name> --runtime=<runtime> --bs=<4K> --rw=<Type of IO> --iodepth=32 --ioengine=libaio --numjobs=no. of jobs (threads)

Drive = /dev/sdb

rw = seq read :: read
seq write :: write
random read :: randomread
random write :: randomwrite
Mixed IO :: rw Seq Read+write
Mixed IO :: randrw :: Random Read + write

name = test name
--bs = Block size
--ioengine = Type of IO engine to use, Default is Libaio

Ex ::
fio --filename=/dev/sdb --name=sample test --runtime=2m --bs=4K --rw=write --ioengine=libaio --iodepth=16 --time_based

- ioengine → It is the parameter defines method or protocol used to issue I/o request to the OS
- libaio → used for high performance SSD's
- iodepth → queue depth

→ max queuedepth, Iops all will depend on Type of i/p we give and protocol we use

SCSI ::

→ SCSI stands for small computer system interconnect/ Interface

→ SCSI has its own command sets called as CDB [Command Descriptor Block]

→ SCSI supports speed up to 8GBPS

→ cable length of SCSI is 6 meters

→ It supports 16 devices

Two types of SCSI ::

① Narrow SCSI :: 0-7 devices

② Wide SCSI :: 8-16 devices

SCSI commands ::

① Read :: Reading from the drive

② write :: write to the drive

③ ABORT :: To stop the previously executed command

④ write Buffer :: To install firmware we use write buffer

⑤ Read Buffer :: To read firmware information

⑥ Request sense :: To check the previously executed command Status

① Read(6)
Read(10)
Read(16)
Read(32)

② write (6)
write (10)
write (16)
write (32)

③ unmap :: To get the location of Garbage data

④ Sanitize :: To remove all the data from SSD

⑤ Send diagnostics :: To send diagnostic data

⑥ Receive diagnostics :: To receive diagnostic data

⑦ Inquiry :: To get the details of SCSI device

⑧ Test unit Ready :: To check device is ready to receive data or not

⑨ Read capacity :: To read capacity of SCSI Device

⑩ format UNIT :: To erase data from SCSI Device [we can ~~remove~~ recover data]

Different types of sanitize ::
① Block Erase :: Block by Block data will be deleted
② Crypto Keys [Cryptographic Erase] :: Removing Cryptokeys

→ sg_sanitize --block -- drive_name
→ sg_sanitize --block --device/dev/sda
→ sg_sanitize --crypto --device /dev/sda

SATA commands ::

Read :: Read from the drive
write :: write from the drive
Sanitize :: To remove all the data from SSD
Secure Erase :: To erase all the data from SATA device
write Buffer :: To install firmware we use write Buffer
Read Buffer :: To read firmware information
TRIM :: Identify the location of deleted data and tell to OS
Identify :: used to Identify device in SATA domain
FPDMA Queued >> NCQ [Native command queuing]

SCSI ::

SCSI passthrough :: To send SATA commands to SCSI HBA
sg_utils ** :: Tool used to send SCSI | ATA commands to the target

sg_read <LBA> <Device_name>

SAS ::

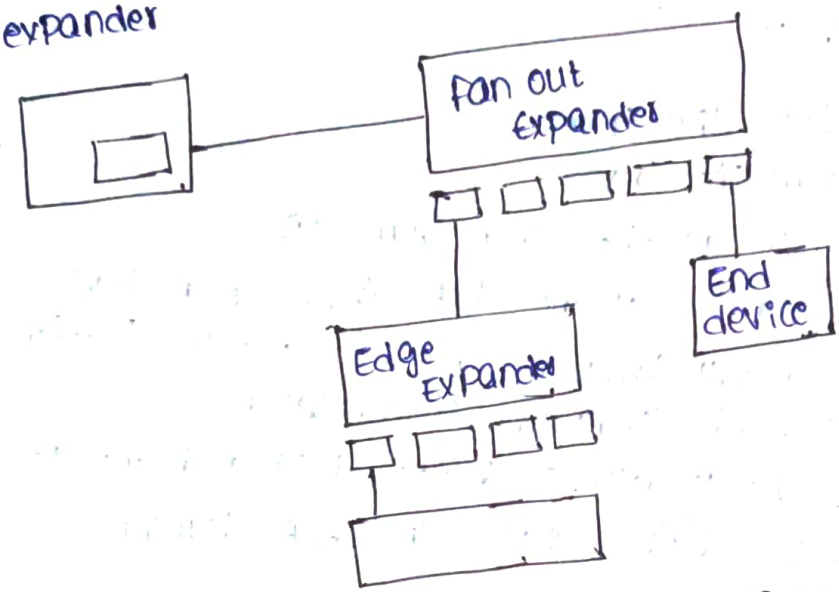
→ SAS stands for Serial Attached SCSI
→ SAS uses the SCSI commands
→ SAS supports speeds up to 12GBPS [24 GBPS is in R&D]
→ It is a serial protocol [follows serial communication]
→ It supports up to 64535 devices
→ It uses the expanders to increase the no. of devices connected

Two types of Expanders ::

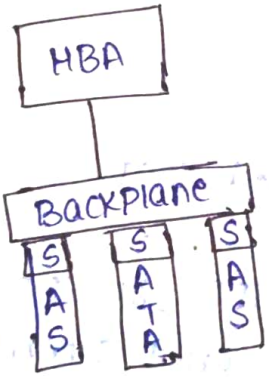
① Fan out Expanders
② Edge Expanders

→ SAS suppats dual ports
→ the rpm of SAS is (rotations per minute) 15000.

Expander is a device which acts like a switch. It is used to extend and connect no. of devices in SAS storage
→ we can connect multiple Edge expanders to one fan out expander



→ SAS Devices are identified as WWN [world wide Name]
→ WWN is a ~~Bit~~ 64 bit Hexa decimal number
→ SAS gives Backward compatibility to SATA devices
→ we can connect SATA drives to the SAS HBA by using Backplane

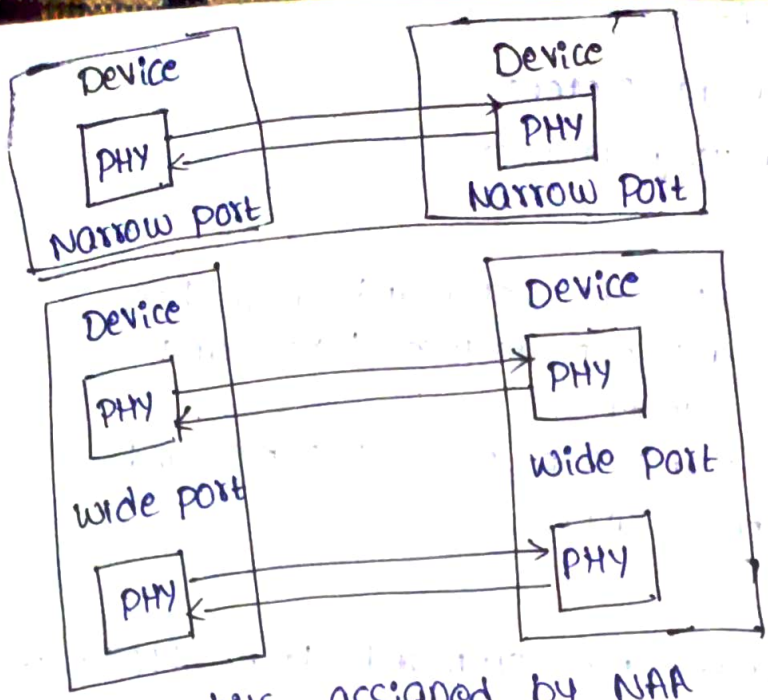


SAS port & PHY ::

→ SAS port contains one or more PHY's and group of PHY's having same SAS Address

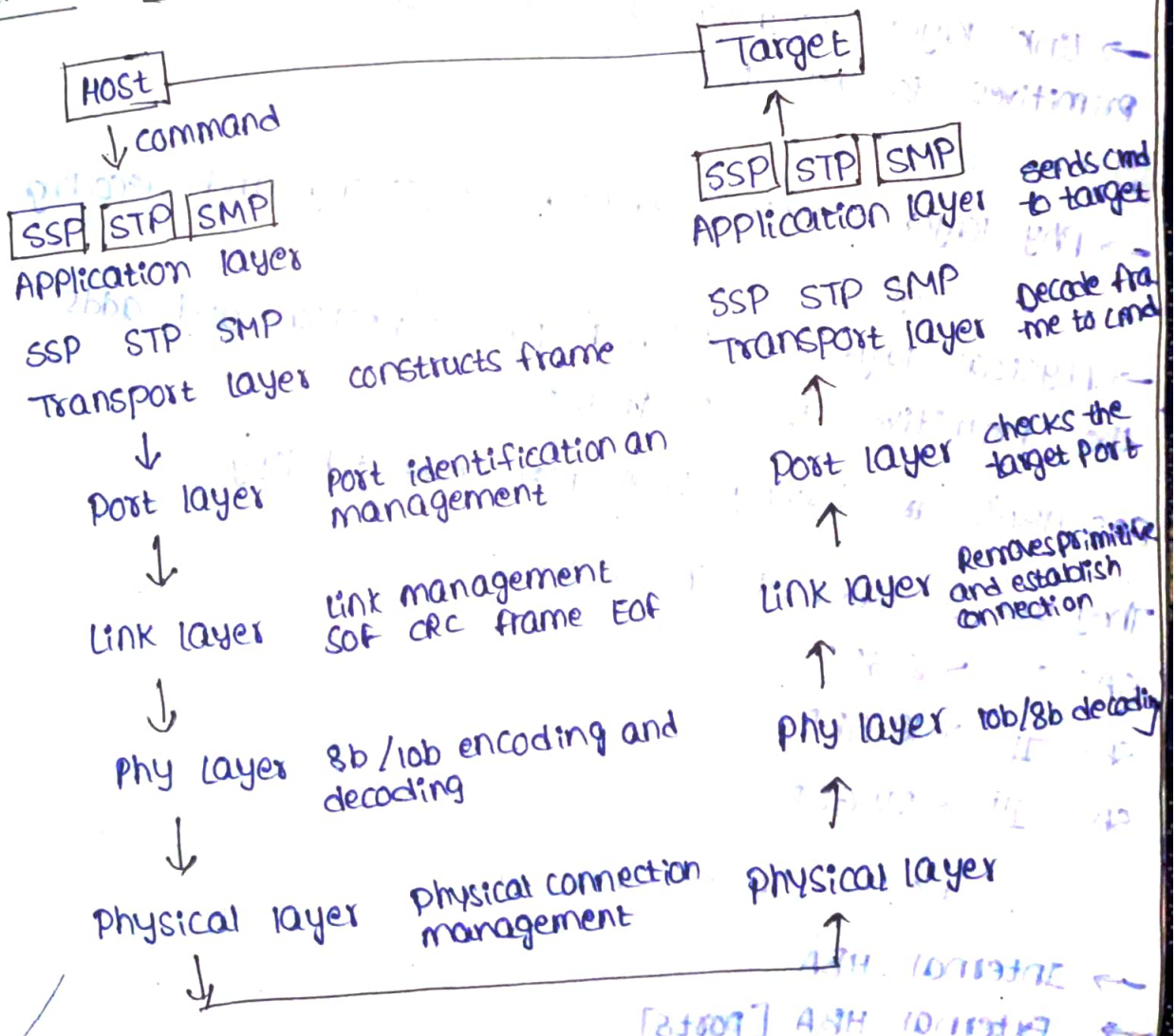
Narrow port :: If there is only one PHY in the port

Wide port :: If there is multiple PHY's in the port



→ SAS 4 bits assigned by NAA
→ 24 bits assigned by IEEE
→ 36 bits assigned by Vendor
→ SAS disks are actually 2.5 & 3.5 Inches in size
→ SAS supports 1 Queue and it can hold upto 256 commands

Architecture of SAS ::



SSP :: Serial SCSI Portocol

STP :: Serial Management protocol

SMP :: Serial ATA Tunnelling protocol

Application layer ::

→ Application layer receives the command from host
→ If it is a SCSI command SSP protocol takes and transfer to Transport layer
→ If it is a ATA command STP protocol takes and transfers to Transport layer.

Transport layer ::

→ Transport layer receives the command from Application layer and construct the frame

Port layer ::

→ Port layer receives the frame transport layer and identifies the end port to which has to send the data

Link layer ::

→ Link layer establish the connection and adds the primitives to the frame

Phy layer ::

→ Phy layer performs 8b/10 bit encoding and decoding

physical layer ::

→ physical layer established the connection and adds the primitives to the frame

SAS HBA's are having dual port compatibility

they are 3 different SAS devices

SAS I > 6 GBPS

SAS II > 12 GBPS

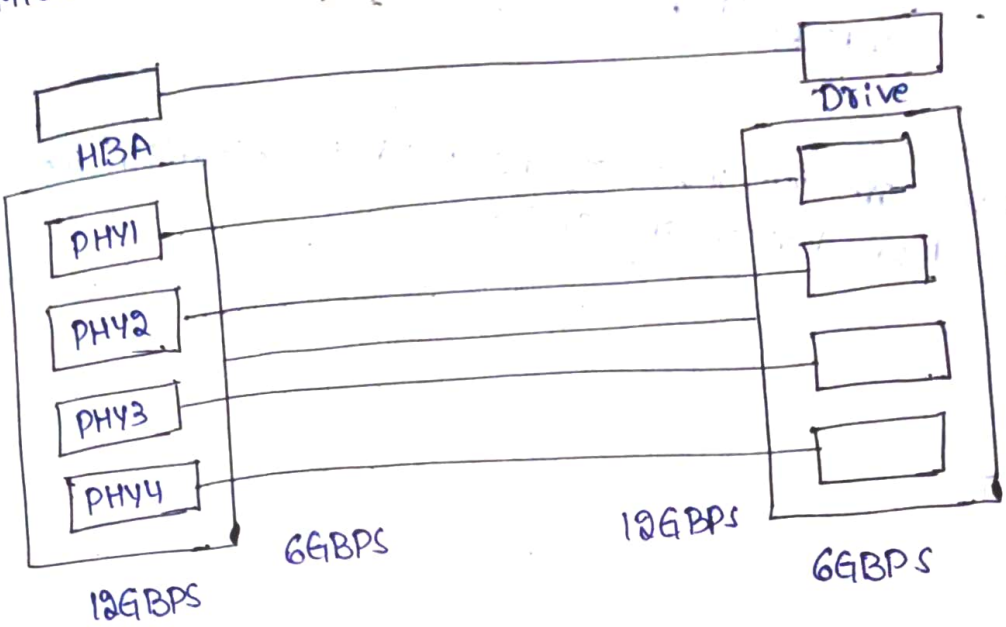
SAS III > 24 GBPS

Different HBA's / RAID controllers ::

→ Internal HBA
→ External HBA [ports]

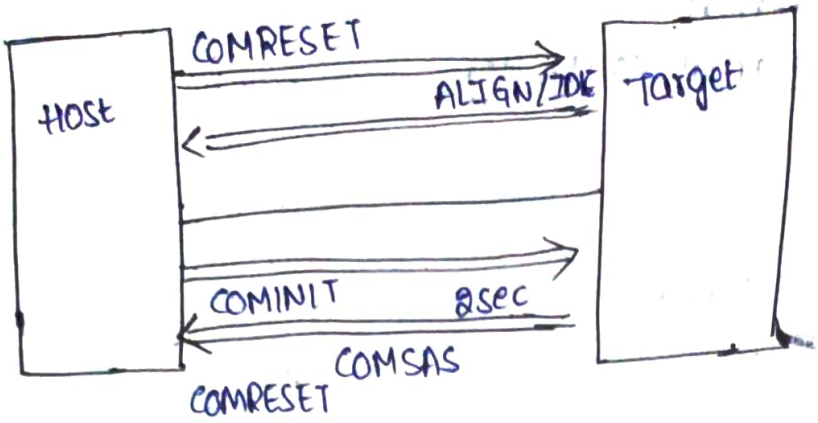
9500 - 8i >> Internal
9500 - 8e >> External controller

Major vendors::
1. Broadcom
2. Microchip / PMC - Cierra



→ 6GBPS >>>>> 6GBPS [Link speed 6GBPS]
→ 12GBPS >>>>> 6GBPS [Link speed 6GBPS]
→ 24GBPS >>>>> 12GBPS [Link speed 12GBPS]

OOB sequence::
→ OOB stands for out-of-Band sequence
→ out-of-Band sequence is a low-level signal that is used to identify device ~~b/w~~ in SAS Domain
→ As soon as we connect device, host will issue the "COMRESET" and the target will respond with "ALIGN/IDLE" Premitives
→ Host will issue the "COMINIT" and target will respond with either "COMSAS" (or) No ~~response~~ primitives
→ If HOST receives the COMSAS then the physical connection b/w SAS and SATAS else the physical connection b/w SAS & SATA



Primitives ::

→ Primitives are low level signals used to establish the connection b/w devices in SAS

1. SOF
2. EOF
3. CRC
4. COMINIT
5. ALIGN
6. IDLE
7. COMWAKE
8. COMSAS
9. xready
10. RReady
11. COMRESET

*Difference b/w SATA and SAS

PCIe Root complex ::

→ It is inbuilt of cpu it will connect cpu to PCIe system

PCIe Switch ::

→ It is used to connect multiple PCIe devices

→ It is having two ports
   1. upstream port
   2. Downstream port

→ upstream port will be connected to root complex

→ Downstream port will be connected to PCI end devices

→ PCI switch and root complex will be connected with

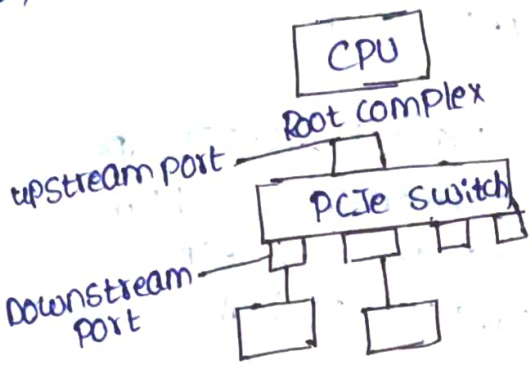
PCIe lanes

PCIe bridge ::

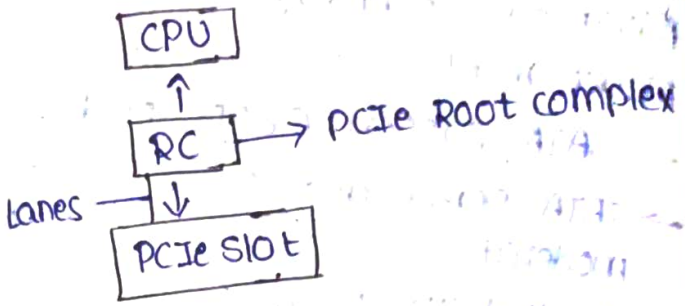
→ from PCIe root complex we can connect to the PCIe bridge used to connect one PCIe bus to another PCIe bus

→ PCIe bridge will convert data from PCI to PCIe (or) PCIe to PCI

→ Root complex will be used to enumerate PCIe devices



Root-complex diagram::



PCI enumeration ::

→ the enumeration is a process of finding PCIe devices (discovering the PCIe devices) and configure into os. so that os can install the drivers and start using it

① System will scan the devices and it will find Bus device function values

BDF (Bus device function)

Ex:: 0 0 1 → BDF

② Reading vendor ID / Device ID

0XFFFF → there is no device

01 → storage [type of device]

08h → NVMe device

③ Assign address

BAR :: Base Address Registers

④ Enable device to OS

⑤ OS will install NVMe driver

HOW CPU will check for PCIe device ?

CPU

↓

check slot

↓

Device found ?

↓

yes / No

↓

check condition

Read Info

↓

Assign memory

↓

Enable device

When PCIe enumeration happens?

→ when new PCIe device is connected
→ when we do hot-plug (removing & re-inserting) the
→ when we restart Server / laptop

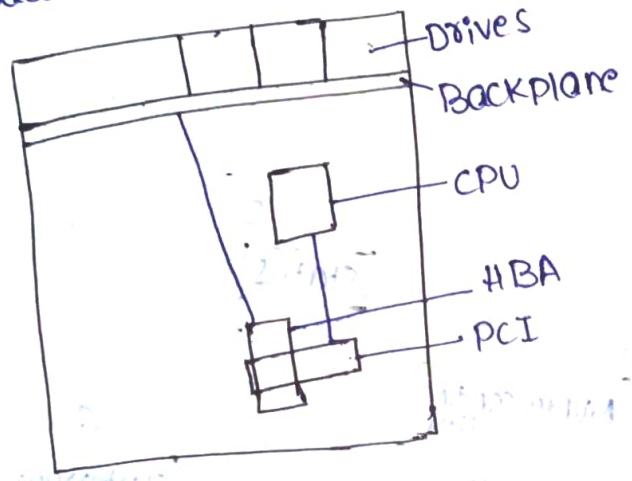
Difference blw SATA and SAS

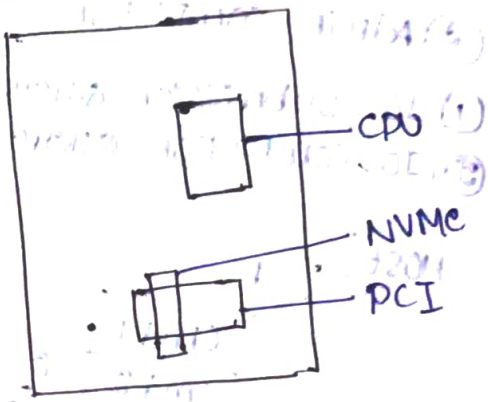
SATA	SAS
→ SATA stands for Serial Advanced Technology Attachement	→ SAS stands for Serial Attached SCSI
→ SATA support 6 GBPS Speed	→ SAS supports 12GBPS Speed
→ SATA performance is moderate	→ SAS performance is high when compare to SATA
→ Reliability is low	→ Reliability is high
→ SATA is used in Desktops, laptops, low-cost storage	→ SAS is used in servers, enterprise storage, data centers
→ SATA uses ATA commands	→ SAS uses SCSI commands
→ SATA supports 8 devices	→ SAS supports 64535 device
→ RPM of SATA is 7200	→ RPM of SAS is 15,000
→ SATA supports 1 queue and it can hold upto 32 commands	→ SAS supports 1 queue and it can hold up to 254 comman
→ SATA is identified by Serial Id	→ SAS is identified by WWN [world wide Name]
→ SATA support single port (one way communication at a time)	→ SAS supports dual port (simultaneous two-way communication)
→ cost of SATA is low	→ cost of SAS is high
→ cable length of SATA is 1 meter	→ cable length of SAS is 10 meters

cost
performance } → high
reliability

NVMe ::

→ NVMe stands for Non-Volatile Memory Express
→ It is one of the disk protocol used to store the data and transfer the data blw two locations
→ NVMe works based on PCIe protocol and it requires 4 PCIe lanes to establish the connection
→ we can connect NVMe devices directly into the PCIe slot
→ PCIe makes NVMe much faster compared to HBA based protocols like SAS and SATA.

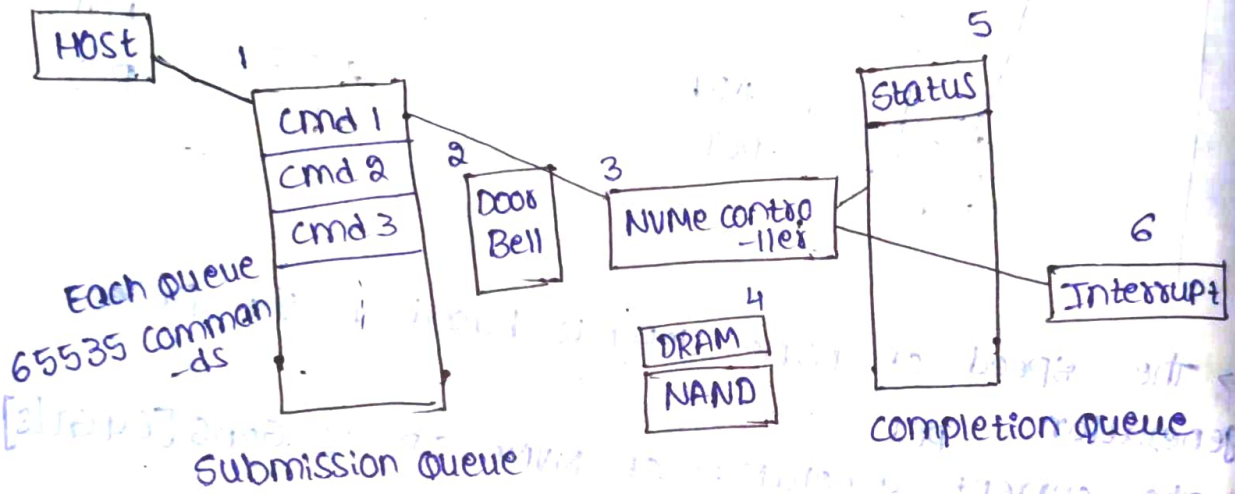




→ the speed of NVMe depends based on the PCIe generation
→ The current generation of NVMe SSD is Gen6 [64 GT/s]
→ NVMe has its own command sets
→ They are two types of commands
  ① Admin commands
  ② IO commands
→ the current release version of NVMe is 2.4.

Why NVMe is much faster than SAS/SATA ::
① PCIe based protocol
② More no. of queues (64K)
③ Capable of parallel transfer commands

→ NVMe works based on Queues
→ It supports maximum of 65535 queues and Each Queue can hold 65535 commands
→ so the Queuedepth of NVMe is 65535 x 65535
→ They are two types of queues
  ① Submission Queue
  ② completion Queue
→ There will be seperate queues for admin commands and IO commands
  ① Admin Submission Queue
  ② Admin completion Queue
  ① IO. Submission Queue
  ② IO completion Queue



→ there will be seperate queues for admin and IO [Submission and completion]
→ All the queues will be sit in HOST RAM
→ NVMe works based on Queues

How NVMe works::
→ HOST writes the data in submission Queue [SQ Entry] and rings the door bell
→ NVMe controller fetches the command from submission Queue
→ NVMe controller executes the command [writes in NAND]
→ NVMe controller writes the status in completion Queue [CQ Entry] and generated the interrupt [MSI-x interrupt]
→ HOST process the command status from completion Queue and releases completion Queue Entry

→ NVMe drives are available as only SSD's
→ NVMe is 10 times faster than SAS
→ Latency of NVMe devices are less than SAS and SATA
→ Throughput of NVMe is more than SAS and SATA
→ IOPS of NVMe is more than SAS and SATA

Admin commands ::

① create IO submission Queue
② Delete IO submission Queue
③ create IO completion Queue
④ Delete IO completion Queue
⑤ Get log page :: To get log pages
⑥ Identify :: To identify the device in NVMe domain
⑦ Abort :: To stop previously executed command
⑧ set features :: To set the features
⑨ Get features :: To get the supported features
⑩ Firmware Image download :: To download firmware image
⑪ Firmware Image Activate :: To activate firmware
⑫ Firmware commit :: To commit the changes
[Before activate we need to send this command]

IO commands ::

① Read :: Reading from flash memory
② write :: writing to flash memory
③ Flush :: To push the data from DRAM to RAM
④ write zeros :: writing zeros in flash memory
⑤ compare :: To compare the data
⑥ NVMe format :: secure erase of data
⑧ AER :: Asynchronous Event Request
→ used to get events from NVMe device

→ To compare the data send from the host and data written in NAND LBR is same or not

→ Initially host will create 1 submission Queue and 1 completion Queue. These Queues will be stored inside host memory
→ Later NVMe controller create remaining Queues. Those Queues will be sit inside NVMe controller and these Queues are part of NVMe subsystem

Namespace ::

→ Namespace is a amount of storage space
→ It is a quantity of non-volatile memory that can be formatted into logical blocks
→ Namespaces can be identified with namespace ID [NSID]

How to create Namespace

Zoned Namespace [ZNS]

→ Namespaces will be managed by NVMe controller

Nvme list

/dev/nvme0	: Actual drive	1TB
/dev/nvme0n1	: Namespace 1	500 GB
/dev/nvme0n2	: " " 2	500 GB

→ To list namespaces
nvme list -ns <drive_name>

→ To use namespace
① create namespace
② Attach "
③ Dettach "

→ nvme create-ns /dev/nvme0 --nsze=1000000 --ncap=1000000

→ nvme attach -ns /dev/nvme0 --namespace-id=1 --controllers=0

→ nvme detach -ns /dev/nvme0 --namespace-id=1 --controllers=0

nvme-cli - used to in Linux/windows OS to manage NVMe device.

SCSI::
CDB:: [Command Descriptor Block]

Read::
- Opcode [Operational code]
- LBA 10 LBA 15 - 10 to 15 LBA
- Transfer length [BS]
- vendor unique [Vendor Specific Data]
- Reserved [00]
- Flag [0 or 1]

Ex:: Read(6) CDB

Read command

Bit Byte	7	6	5	4	3	2	1	0
0	Operation code (08h)							
1	Logical Unit Number			Logical BLOCK Address (MSB)				
2	Logical BLOCK Address							
3	Logical BLOCK Address (LSB)							
4	Transfer Length							
5	Vendor unique			Reserved			Flag	Link

class: 18, 19, 21, 23

SSD features::

→ Erase Blocks.
→ wear Leveling.
→ Write Amplification.
→ Garbage collection.
→ Ecc
→ over provisioning.
→ FTL / LBA.
→ DWPD
→ Endurence.

→ TBW
→ MTBF
→ P/E cycles.
→ P List
→ G List
→ S.M.A.R.T

Erase Blocks ::

→ In SSD Erase happens in a block level

→ In SSD we need to Erase the data before rewritting
→ write operation happens in "page level" and Erase operation happens in "block level"

why?

→ SSD controller has to apply more voltage for Erase operation compared to the write/Read operation
→ If SSD controller erases the individual page there are high chances for next page to get effected so SSD controller will erase the data in block level this feature is called as Erase Blocks

Wear Leveling ::

→ If SSD controller continuously Read/write (or) Erase the same cell again and again there are high chances for that particular cell/page to wear out (go bad) before the remaining cells. this will impact the lifespan of SSD.
→ so SSD controller will make sure that RAID's are distributing among equally all the cells in SSD. This feature is called as wear leveling.

Advantage ::

→ Lifespan of SSD will improve.

Write Amplification ::

→ In SSD the data ~~written~~ written to the NAND will always be more than the data sent from the host

→ Write Amplification = $\frac{\text{Data written to the NAND}}{\text{Data sent from the host}}$

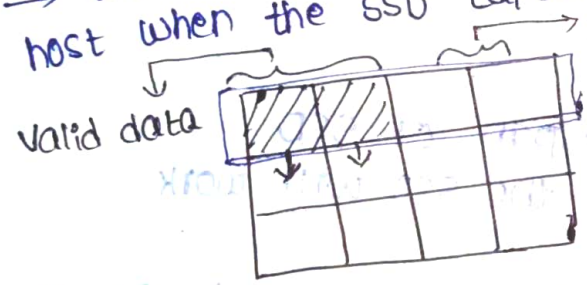
→ Write Amplification is always present in flash memory
→ we cannot completely eliminate it from flash memory

Garbage collection::

→ In SSD write operation happens in "page level" and Erase operation happens in "block level"
→ In order to erase the entire block SSD controller will move good pages to next available block and it will erase the entire block. this process is called as Garbage collection.
→ It is automatic process in SSD and SSD controller is responsible for performing GC (Garbage collection).

TRIM/UNMAP::

→ These commands will notify SSD controller about the location of stale pages which are not required for operating system
→ these commands will be automatically issued from the host when the SSD capacity is 80% filled



ECC::[Error correction code]

→ ECC stands for Error correction code which is used to detect and correct the errors in SSD

Ex:: XOR algorithm
Hamming code
Read solomen Algorithm
BCH

They are two types of errors
① correctable errors
② uncorrectable errors

① Ex:: Parity error
② Ex:: Hardware error (UE)

FTL::

→ FTL stands for flash Translation layer
→ FTL will map logical block address (LBA) to the NAND physical block Address (PBA)

→ FTL is the part of SSD controller and it is also responsible for performing operations like wear leveling, Garbage collections

LBA::

→ LBA stands for Logical Block Address
→ It is used by the host to identify and access the physical Blocks in SSD
→ Host will allocate the LBA's for each block in SSD

DWPD::

→ DWPD stands for Data writes per Day
→ It is a measure of SSD Endurance
→ It will tell us how many times we can write the full capacity of SSD per Day

Endurance:: Ex:: 3x1TB = In a single day 1TB capacity drive can be written 3 times of its capacity

$$DWPD = \frac{\text{Total bytes written in a life time of SSD}}{\text{Drive capacity} \times \text{Warranty Days}}$$

→ DWPD of enterprise SSD is 3 to 10
→ DWPD of client SSD is 0.1 to 1

Endurance::

→ Endurance refers to the lifespan of SSD
→ It will tell us how long the SSD will work under the right work load
→ In modern SSD's the life span of SSD will be 5 years (After 5 years SSD flash cells may wearout)
→ SLC has more endurance compare to other cells

TBW::

→ TBW stands for Total Bytes written
→ Ex:: 500 TB
In SSD life span 500 TB of data written to NAND

TBW = DWPD * Drive power on Days

P/E cycles::

→ P/E cycles stands for program/Erase cycles

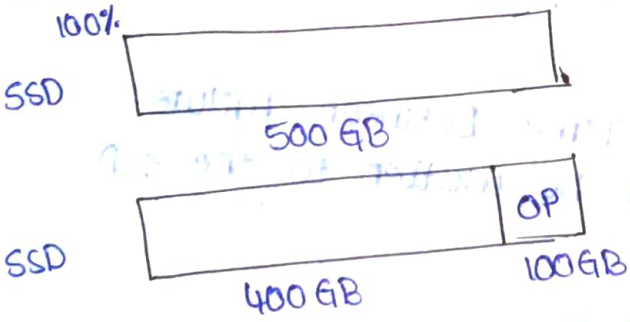
→ No. of read / write, erase operation can be perform on SSD

S.M.A.R.T ::

→ S.M.A.R.T stands for self Monitoring and Analysis Reporting Technology
→ which is used to monitor the health status of HDD/SSD
→ we can get Different SMART attributes like
① Power on Hours
② Power cycle count
③ Total Bytes written
④ Hardware Error [UECC]
⑤ wear leveling count
⑥ CRC Errors
etc....
→ we can also get firmware version, total capacity, serial number, model etc...
→ To get SMART data in linux
smartctl -a <drive>

over provisioning ::

→ In SSD we can reserve some storage space for over provisiong. This space can be visible to only SSD controller and host cannot see/access the over provisionin
-g storage



→ SSD controller will use OP storage for operations like Garbage collection
→ we can perform set OP and clear OP operations from SSD

Glist :: [Run time errors]

→ Glist stands for Grown Defect List

→ these kind of errors will increase as long as we use the SSD

→ It is a list of bad blocks that develop during the SSD usage over time [Run time]

→

P List ::

→ Plist stands for permanent defect list which are identified during the SSD manufacturing for the factory [Manufacturing Defects]

Logic.

If we have logical errors / hardware & manufacturing errors which type of errors we may see on all the samples [Drives]

Logical / Firmware ::

Ex :: Logic Implemented in firmware / software

Ex :: 3+2=6

Hardware / Manufacturing Error ::

During the manufacting of sample

Ans :: Manufacturing Errors may or may not occur on all the samples

→ Logical Errors can be seen on all the samples as the logical implementation is wrong

MTBF ::

→ MTBF Stands for Mean Time Between Failure

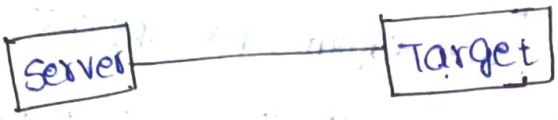
→ the amount of data can be written to the SSD before failure

Storage Topologies ::

1. DAS
2. SAN
3. NAS

1. DAS ::

→ DAS stands for Direct Attached Storage
→ Here we will connect storage and servers directly with the help of cables
→ Shared storage is not possible in DAS
→ It supports protocols like SAS, SATA, NVME



HOST
DAS

TARGET
SAS, SATA, NVME

→ DAS is a block level storage

Advantages::

1. faster
2. Easy to implement and maintance

Disadvantages::

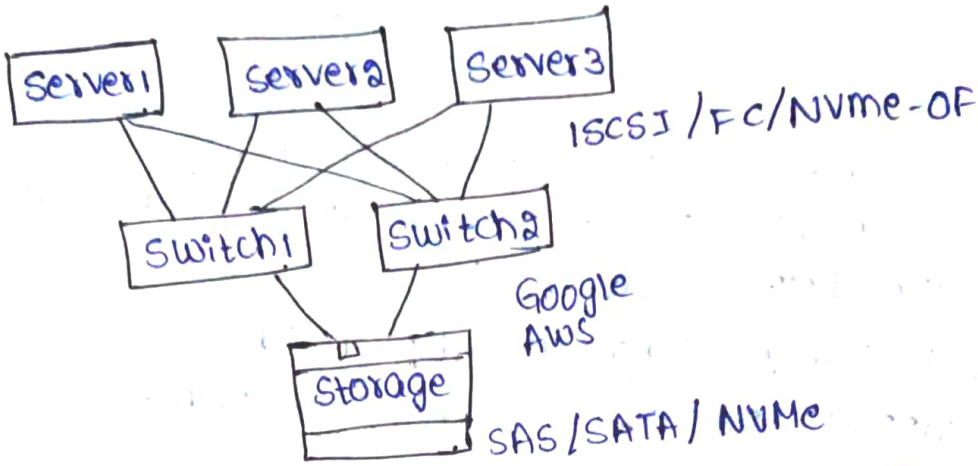
1. No shared storage
2. No redundancy for servers

2. SAN::

→ SAN stands for Storage Area Network
→ Here servers will be connected to the storage with the help of switches
→ SAN supports shared storage and remote storage
→ SAN is a Block level storage
→ It supports protocols like ISCSI, FC, NVME -OF ~~(or) fabric~~
→ SAN has a redundancy for hardware components like switches, Bridges and servers

→ In SAN multiple users can access ~~multiple users~~ storage at a time

→ SAN supports features like multipathing, zoning and high availability.



↘ Same diagram for NAS also

Block level Storage ::

→ In Block level storage we will create RAID/Volume and attach raw storage space [Raw volume without File system] to the user / server

→ server / user can see storage space [volumes] as local storage and he can format with his required file system

Ex:: DAS, SAN

File level Storage ::

→ In file level storage we will create RAID / Volume and apply file system at the storage side and share the storage across the network as files

→ It supports protocols like NFS and CIFS

Ex:: NAS

③ NAS ::

→ NAS stands for Network Attached Storage

→ It supports file level storage

→ server and storage connected with multiple switches

→ NAS also provides redundancy for hardware compo-nents

→ NAS supports protocols like NFS (Network file system) CIFS (common internet file system) → windows OS
↓
Linux Based OS

ISCSI::

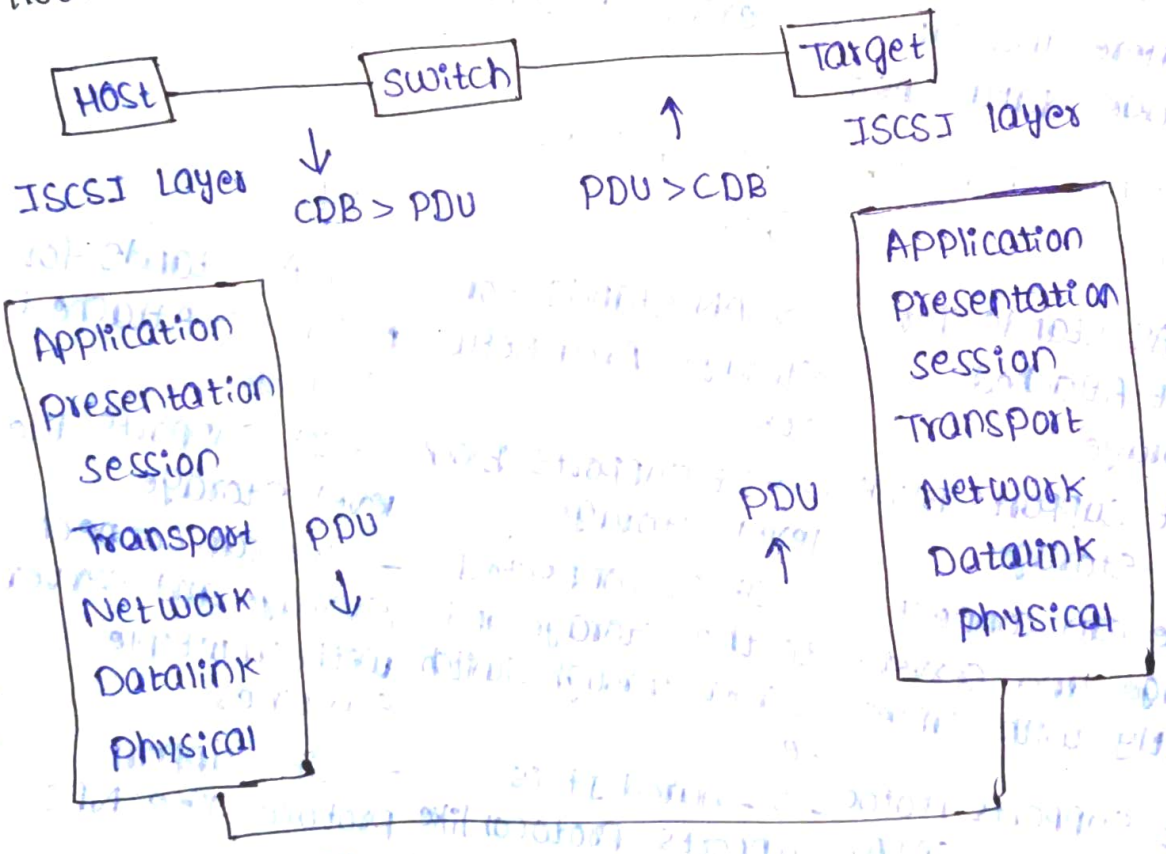
→ ISCSI stands for Internet SCSI
→ It works based on TCP/IP protocol
→ ISCSI uses SCSI commands and it will transfer SCSI across TCP/IP network
→ ISCSI supports speed upto 10GBPS
→ they are two type of ISCSI
① Software ISCSI
② Hardware ISCSI
→ for software ISCSI we need to install software based ISCSI initiator and target
→ for hardware ISCSI we need to use hardware based ISCSI HBA's at the host and target
→ Here we can use existing LAN cables to transfer storage data also
↓
RJ45

Difference blw DAS, SAN and NAS

DAS	SAN	NAS
→ DAS stands for Direct Attached Storage	→ SAN stands for Storage Area Network	→ NAS stands for Network Attached Storage
→ It supports Block level Storage	→ It supports Block level storage	→ It supports file level Storage
→ we can connect storage and servers directly with cables	→ It is connected to the storage and server through Switches	→ we can connect storage and server with multiple switches
→ It supports protocols like ATA, SATA, SCSI, SAS, NVMe	→ ~~shared~~ It is supports protocol like ISCSI, FC, NVME-of	→ It supports protocol like NFS, CIFS
→ Shared storage is not available in DAS	→ shared storage is available	→ shared storage available

→ It is a local storage	→ It is a remote storage	→ It is a remote storage
→ Implementation of DAS is simple	→ Implementation of SAN is costlier than DAS	→ Implementation of NAS is costlier
→ there will be no redundancy for hardware components	→ SAN provides redundancy	→ NAS also provides redundancy
→ DAS supports features like hot-plug RAID supports, smart monitoring of drives	→ SAN supports features like zoning, HA multipathing making De-Duplication	→ NAS support features like file sharing, snapshot replication HA
→ DAS is faster than SAN, DAS	→ SAN is slower than DAS	→ NAS is slower than DAS
→ Latency is less performance is more	→ Latency is more → high performance	→ Latency is more → medium Performance

How the communication happens in ISCSI



→ ISCSI layer receives the command [CDB] from host and converts to PDU [Protocol Data Unit] and transfer across the TCP/IP Network

→ At the target side ISCSI layer converts back the pdu to CDB and sends to the target

→ ISCSI are Identified IQN [ISCSI qualified Name]

IQN format :: iqn.1991-05.com.microsoft:hp

- iqn.1991-05 → manufacture Date
- com.microsoft → company name
- hp → vendor specific

ISCSI Discovery process::

→ In ISCSI we need to discover host from the target and target from the host

Windows:: Step-1::

① Login to ISCSI Initiator
② click on targets
③ Enter target name and click on quick connect
④ click on discovery and list all the available targets
⑤ select the required targets
⑥ select ISNS servers (DNS for ISCSI) and click on add server

Step-2::

Verify ISCSI connections, ping host from the target and target from the host if both host and target pinging from each other than successful connection is establish

Step-3::

Mapping the volumes to the host

① Login the Storage array
② create RAID group
③ create volumes
④ And map the volumes to discover the host by using its IQN / IP address / Host Name

Linux::

iscsiadm -m discovery -t sendtargets -p <ip_address>
iscsiadm -m node -l [To login all discovered targets]
iscsiadm -m node -T <targetname> -p <ip_address>
[To login particular target]
Ping <HOST>
Ping <target>

Volume mapping::

iscsiadm -m node -u [To login all discovered targets]
iscsiadm -m node -T <targetname> -p <ip_address>
[To login particular target]

In windows: Step① : Discover the target
Step② : connect to the target
Step③ : create the session
Step④ : Transfer the data

① Discovery
iscsiadm -m discovery -t send targets -p <target_ip>

② Login phase
Initiator connects to target using TCP
iscsiadm -m node -T <target_iqn> -p <target_ip> --login

③ session establishment
Session creates b/w host and target
iscsiadm -m session (verify connection)

④ command execution (data transfer)

Log out::
sudo iscsiadm -m node -T <target_iqn> -p <target_ip>
--logout

Rescan
iscsiadm -m session --rescan

After Discovery
Ping :: we will ping host from target and target from the host
→ If both are pinging successful iscsi connection is established

FC ::

→ FC stands for fibre channel
→ FC also uses SCSI commands
→ FC devices are identified by WWN [World Wide Name]
→ It supports speeds upto 128 GBPS [32GBPS FC]
→ FC HBA contains multiple ports
→ ports will be identified by WWPN [World Wide Port Name]
→ FC is faster than ISCSI
→ To implement FC storage Network we need FC HBA, FC switch, FC Storage
→ HBA > Switch > Storage Array

FC layers ::

FC4 :: Protocol Mapping
FC3 :: port layer - Transfer the data from port
Enable services > Multipathing, RAID
FC2 :: frame Definition :: SOF payload CRC EOF
FC1 :: Encoding / Decoding :: 8b/10b Data,
FC0 :: physical connection

FC4 :: protocol Mapping layer
To map the protocols
1. FCP - SCSI
2. NVMe-OF

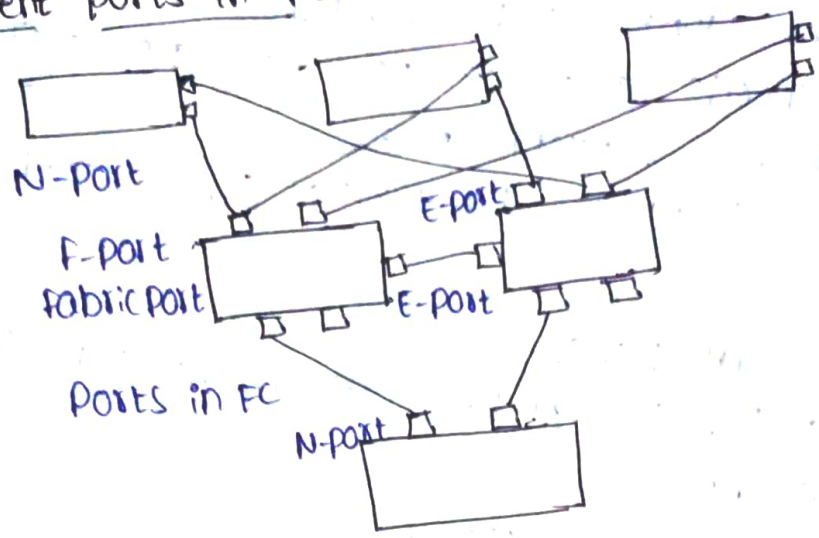
FC-3 :: common service layer
Different services support
1. RAID
2. Multi pathing
3. Split data over multiple ports / links

FC-2 :: protocol / data frame layer
create frames
Defines FC Topologies
Frame format :: SOF FrameHeader payload CRC EOF

FC-1 :: Encoding / signaling
8b/10 encoding

FC0 :: physical layer
for physical connection

Different ports in FC::



1. N-Port > Node Port > Server HBA Port
2. F-Port > Fabric Port > Switch Port connecting to N Port
3. E-Port > Expansion Port > switch to switch

F Logi > HBA talks to Switch
P Logi > Host Port to Target Port

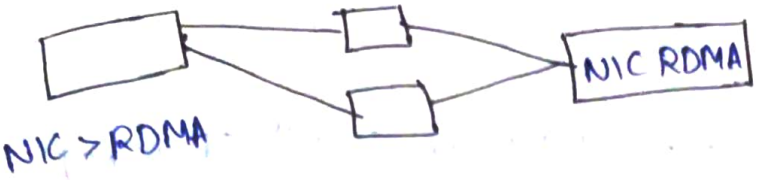
NVME-OF::

In NVME over fabric we can transfer the NVME command over the fibre channel Network
→ Here we can use the advantages of NVME protocol (low latecy, high throughput, high IO) in SAN Topology

they are different ways of implement NVME fabric
① NVME over RDMA

In NVME-Of uses RDMA protocol to establish the connection b/w host and target
→ we need to insert NIC cards with RDMA enable at host and target
→ In RDMA [Remote Direct Memory Access] the communication happens b/w memory to memory so there will be less burden on cpc and it has more performance

→ NVMe-OF eliminates the latency issue that are there is FC and ISCSI



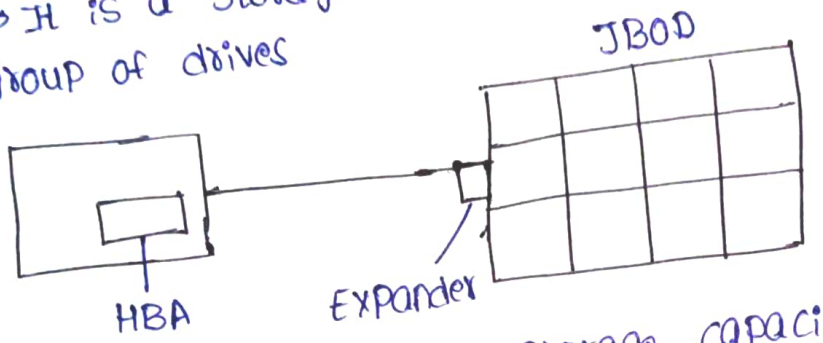
NIC > RDMA

Types of Storage Devices::

① HDD
② SSD
③ JBOD
④ Storage Array / EBOD [Extended bunch of disks]

JBOD::

→ JBOD stands for Just bunch of disks
→ It is a storage enclosure [box] where it contains group of drives



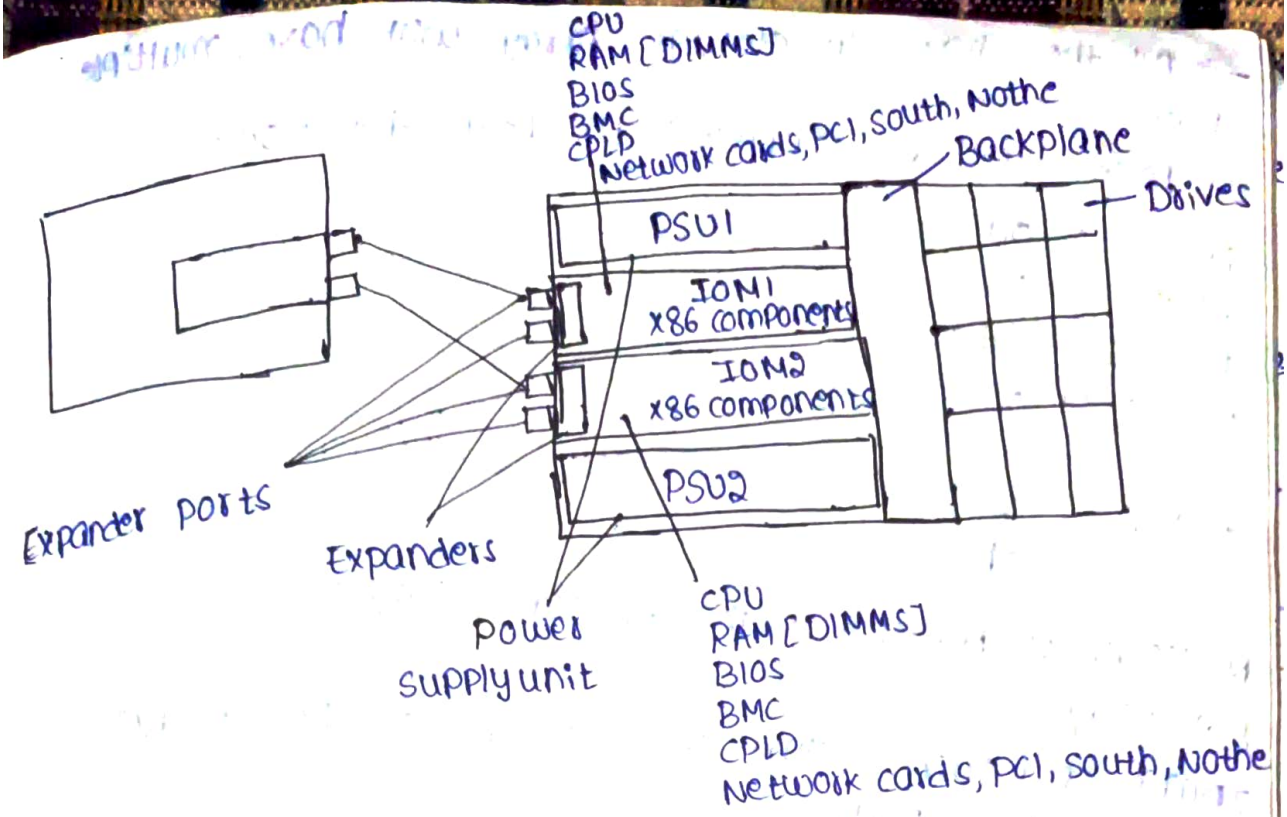
→ It provides additional storage capacity to the server
→ JBOD is not an intelligent device
→ we need to connect JBOD with HBA to make it as intelligent device
→ JBOD contains expander and drive slots
→ OS can directly discover all the drives in JBOD and drives will be visible to OS as a individual drives

components of JBOD::

① Expander
② Drive slot
③ Power supply unit (PSU)
④ fans

Storage Array::

→ Storage Array is a device and it can hold group of drives
→ Storage Array contains operating system and x86 components [CPU, RAM, BIOS, BMC, CPLD]
→ It contains no. of nodes / IOMS / controllers



components of Storage Array::

① Drives
② Backplane
③ IOM's / controllers / Nodes / Modules
components of IOM / Node / Module
CPU, RAM, ROM, Expander, Bios, BMC, CPLD, Expanders, Network ports, South bridge, North bridge, DIMMS, PCI SLOTS
④ PSU
⑤ Network ports

Features of Storage Array::

① Multipath
② HA / cluster
③ Takeover and given back / failover and fail back.
④ Remote management
⑤ RAID / volumes
⑥ Snapshots
⑦ Remote copy / Replication / Deduplication
⑧ LUN / LUN Mapping / LUN Masking / Zoning

→ All the drives in storage Array will have multiple paths
→ Drives will be connected to both the nodes

Node1 :: 0a
Node2 :: 0b
Drive Io path::
Drive1 :: 0a.1
0b.1
Drive2 :: 0a.2
0b.2

Multipath :: Multiple paths to same target
Advantages ::
Incase of 1 Path failure we can access the storage from another path

cluster ::
→ Group of Nodes in storage Array
→ we need to create cluster, add all the nodes in a storage array and enable high availability [HA]

High availability [HA] ::
→ It is a cluster level feature
→ To provide continuos access of storage to the user we need to create cluster
→ Enable high availability in cluster level

To achieve HA::
① Multipath
② All the Nodes should be in a same cluster

Different storage configurations ::
① Active to Active
② Active to passive

→ these configurations are used in storage clusters

① Active to Active ::
→ In active to active configuration both the nodes will be active and transfer the data at

same time
→ we can see more performance (throughput)
→ If one IOM fails (controller) the entire data will transfer from active IO channel

Advantages ::
→ High throughput

Disadvantages ::
→ chances of hardware failure

② Active to passive ::
→ In Active to passive storage configuration, one node will be active and acts like master and another node will be inactive and acts like slave
→ In active to passive configuration data will be transfer from only one path at a time
→ If master fails, slave IOM will take over the IO channel and transfer the data
→ once the master IOM comes back slave IOM will give back the IO channel to master IOM
→ this process is also called as failed over and failed back

RAID Groups ::
→ Group of physical drives into logical drive

volume ::
→ small amount of storage space from RAID Group
→ we can create upto 32 volumes for a RAID group

TO create volume ::

```
Volume create --name <vol_name> --raid-group
  <RAID_Group> -- Size = <Size of volume>
```

Volume show ::
Volume 1 >> LUN # >> Logical unit Number >> 0
Volume 2 >> LUN 1

LUN Mapping :: Mapping the volume to the particular target [server/user]

How to map the ~~volume~~ LUN :: Based on WWN / IQN of the host

Volume map vol_name | LUNID --target <IQN | WWN>
--permissions <write permissions>

ISCSI :: IQN
FC :: WWN
SAS :: WWN

LUN Masking ::

→ Making the LUN / volume invisible to the particular target
→ To ~~restrict~~

Remote Management ::

① SSH
② BMC
③ serial console port [using rconsole] :: we can access Nodes without OS, network, when nodes are rebooting also.

Snapshot ::

→ point in time copy of a volume
→ we can create upto 32 snapshots of a volume
→ snapshot size will be less than volume size

Replication :: It is nothing but copy the volume

① Synchronous Replication
② Asynchronous Replication

① :: Data will be return to primary and secondary volume at the same time

Adv ::
→ No data loss

Disadv ::
→ low performance

② :: Data will be first return to primary volume then it transfer to secondary volume

Adv ::
→ Data transfer is faster

Disadv::
→ possible data loss incase of ~~drive~~ hardware fails

Remote copy:: same
→ copying the data in two different geographical locations in case of any disasters we can recover the data
→ It is mainly useful for Disaster Recovery (DR)

Deduplication::
→ Removing duplicate data by writing only unique data into storage
→ It will replace all the duplicate data with pointers
100GB 2 to 5GB > 10001
500 to 503GB > 10001 > pointer

Zoning::
→ Dividing the SAN fabric into multiple zones ~~once~~ after we create zones ~~of~~ only that particular zone servers can communicate with each other
→ one zone server cannot communicate with other zone server
→ they are 2 types of zoning
① Soft zoning
② Hard zoning

① Soft zoning:: It is created at HBA level by using WWN of HBA
② Hard zoning:: It is created at switch level by using WWPN of switch ports

Thin provisioning:: Logically allocates the complete data but physical storage will be assigned whenever controller writes the data
Thick provisioning:: Assigning the complete physical storage dedicated to that particular host

HDD::

Test Scenarious:

→ verify HDD is detected by os. } ✓
→ verify Read/write is

SSD::

→ verify SSD is detected by os. } ✓

HDD::

→ verify HDD is detected in BIOS/UEFI.
① power ON system
② open BIOS/UEFI settings
③ Navigate to storage section

Result :: HDD should be listed with correct model and capacity

→ verify HDD is listed in OS (lsblk)
① Boot OS
② Run command lsblk / open disk management
③ check if HDD is listed

Result :: HDD appears with correct name, size, partitions

NVMe ::

To update firmware

three steps ::

① firmware download
② " Activate
③ " commit

firmware update procedure ::

→ we get drive firmware in .bin format

-nvme fw-download /dev/nvme0 --fw=/path/to/firmware.bin

-nvme fw-activate /dev/nvme0

-nvme fw-commit /dev/nvme0 --slot=1 --action=1

Different commit actions;

Action		Meaning
0	-	Replace image only (no activation)
1	-	Activate at next reset
2	-	Activate immediately (no reset)
3	-	" " + reset
6	-	Replace + activate at next reset

To get firmware version

nvme id-ctrl /dev/nvme0

NVMe CLI ::

→ used to manage and configure NVMe devices

nvme list > To list all nvme drives

nvme smart-log /dev/nvme0 > get log page

nvme get-feature /dev/nvme0 -f 0x07

∴ To get no. of queues

nvme set-feature /dev/nvme0 -f 0x07 -v4

∴ To set features

→ To create Io queues

nvme create-cq /dev/nvme0 --cqid=1 --qsize=64

→ nvme create -sq /dev/nvme0 --sqid=1 --cqid=1
--qsize=64

→ Delete IO queues
nvme delete -sq /dev/nvme 0 --sqid=1
" " -cq " --cqid=1

→ Abort command
nvme abort /dev/nvme 0 --sqid=1 --cid=123

→ format
nvme format /dev/nvme on1

IO commands ::

nvme read /dev/nvme on1 --start-block=0
--data-size=4096

nvme write /dev/nvme on1 --start-block=0
--data-size=4096

nvme flush /dev/nvme on1
nvme write-zeroes /dev/nvme on1 --start-block=0
--block-count=100

identify → nvme id -ctrl

ISCSI discovery process :: (targets configuration discovery)

① login to ISCSI initiator
② click on targets
③ go to configuration it will show initiator name
④ copy the initiator name
⑤ go to targets
⑥ paste the url in targets then quick connect
⑦ then done
⑧ go to discovery you will see the url
⑨ In ~~ISCSI~~ isns server click on add server
⑩ enter the ip and click on ok

verify iscsi connection ping host from target and target from host if both target and host pinging

each other successful connection is established

LUN::
→ It stands for logical unit Number
→ After creating a volume a unique number will be assigned to each volume that is called LUN
→ LUN always start with '0'
→ we identify the volume by its Name (or) LUN

Scalability (or) cascading (or) Daisy chain mechanism::
→ we can use the storage capacity by connecting no. of JBOD's to storage Array
→ we can connect upto 24 JBODs to storage Array

deduplication : Types:
① file-level :: check duplicate files
② Block-level :: " " Blocks (Hitachi vsp uses this)

compression : It reduce size of each block does not remove duplicate blocks

copyback :: If ~~drive~~ failure ~~in~~ R1

metadata :: meta data will be stored in the RAID group it contain information about RAID group such as
RAID level
capacity of the RAID group
Block size
RAID group name/ID
→ RAID controller will use meta data to identify the RAID group

⇒ usecases of all RAID levels
where we will use
Test scenarios on all RAID levels

Hot Swappable / Hot Pluggable ::

when everything is up and running we can remove and reinsert the drives from RAID group is called as Hot Swappable / Hot pluggable

consistency check :: (CC) (BCC)

→ It is a background operation it will verify the consistency of the data and it will fix the parity errors for mirror inconsistency
→ RAID controller will periodically check for the Parity errors

How consistency check works ::

RAID controller read all the blocks

↓

find parity consistency errors

↓

fix the parity or mirroring errors

→ there will be a moderate performance impact when cc happens

patrol read ::

→ It is the Background operation
→ RAID controller will scan every sector/Page on a physical drive to detect weak sectors / pages
→ If any weak sector found it will fix the weak sectors
→ Patrol read will be performed on individual drive level those are connected to RAID controller

Hardware RAID ::

vendors ↔ Broadcom (LSI)
↔ microchip

Broadcom → 9400 → HBA , 9440 → RAID controller
9460 → RAID controller
9500 → HBA
9560 → RAID controller
9600 → HBA
9660 → RAID controller

8i, 16i → Internal card
8e, 16e → External card

→ Internal card will be connected to server
→ External card will be connected to JBOD / Storage Enclosure

we need to use vendor provided utility to manage hardware RAID
→ storcli or megaraid storage manager
→ To list the drives connected in HW RAID
./storcli /c0 show
All controller information

To create HW RAID

Syntax::
storcli /c<controller> add VD type=raid<level> drives = <enclosure>:<slot>, <enclosure>:<slot>

Ex::
storcli /c0 add vd type=raid1 drives = e129:0, e129:1

CID → 0, Slot ID → 0,1, enc ID → 129

To add hotspare → storcli /c0 add hotsparedrive=129:5
To perform ORCE → storcli /c0 /v0 start migrate option=add drives=129:4
" " ORLM → storcli /c0 /v0 start migrate type=85.
To start CC → storcli /c0 /v0 start cc
Patrol read → storcli /c0 start patrolread
To use capacity of drive → storcli /c0 /e129 /s5 set good

Drive modes in HW RAID::
Two modes → JBOD :: OS can discover the drive directly
UGOOD :: Drive connected to RAID controller but not part of any RAID Group
ONLINE :: Drive is part of RAID Group
↳ state

⇒ storcli /cx[/ex] /sx set jbod
→ lsblk :: To list drives
⇒ storcli /cx[/ex] /sx set good

PLP (Power loss protection) ::

Whenever there is a power loss in SSD the data in the DRAM will be pushed to NAND. Technically we call it as SPOR (Sudden Power on RESET)

DRAM less SSD ::

Adv :: Lower cost

Disadv :: slower IO operation

firmware ::

→ It is a type of software that runs on hardware
→ firmware defines the functionality of hardware
→ without firmware hardware devices won't work
→ firmware is a developer code (C, C++) that is converted to a .bin file and release to testers or custo to install it on the hardware

when developers will release the firmware to testers ?

→ when the product is new (Testing for the first time
→ when there is a new feature added
→ whenever there is a change in code (BUG / defect fi

Device drivers ::

→ device drivers is a software that establish the communication b/w os and hardware.
→ we need to install device drivers on top of hardware from the operating system
→ Device drivers will reside inside the os
→ without device driver os cannot recognize the hardware
→ diff b/w device driver and firmware